

# Holographic Self-Oracularization for Boolean Satisfiability: A Fourier–IFS Framework for the P versus NP Problem

Kaoru Aguilera Katayama

September 22, 2026

**Official repository:** [github.com/PolloXDDD/OMEGA-Fourier-IFS-SAT-Engine](https://github.com/PolloXDDD/OMEGA-Fourier-IFS-SAT-Engine)

## Abstract

This manuscript develops an explicit framework for Boolean satisfiability through recursive self-oracularization. The construction begins by viewing NP witness search as a problem of missing directional information, introduces a meta-search architecture in which oracle requests are recursively represented inside the same problem class, and gives a finite “holographic” closure of the resulting recursion. Iterated function systems provide the finite self-similar generator, while Fourier–Walsh transfer operators provide a global spectral representation. For 3-SAT, the paper derives an exact harmonic oracle field whose positivity characterizes whether a partial assignment has a satisfying completion, gives a clause-local Walsh transfer circuit, and describes a weighted IFS path representation of the resulting spectral expansion. A reflective layer applies the same compression architecture to proof obligations about the quotient construction. The resulting closure theorem isolates the exact structural hypotheses under which the construction decides 3-SAT in deterministic polynomial time and therefore yields  $\mathbf{P} = \mathbf{NP}$ . The paper concludes with an executable Google Colab implementation that reads DIMACS CNF files and returns verified SAT assignments or UNSAT results.

## Contents

|                                                                       |          |
|-----------------------------------------------------------------------|----------|
| <b>Roadmap</b>                                                        | <b>5</b> |
| <b>1 NP-Complete Problems as Blind Witness Search</b>                 | <b>5</b> |
| 1.1 Motivating Perspective . . . . .                                  | 5        |
| 1.2 Polynomial Verification Relations . . . . .                       | 5        |
| 1.3 Blind Witness Search . . . . .                                    | 6        |
| 1.4 Guided Witness Search . . . . .                                   | 6        |
| 1.5 The Blindness Gap . . . . .                                       | 7        |
| 1.6 NP-Complete Problems from the Blind-Search Perspective . . . . .  | 7        |
| 1.7 The Directionalization Principle . . . . .                        | 8        |
| <b>2 Levin Universal Search as an NP-Complete Meta-Search Problem</b> | <b>8</b> |
| 2.1 The Existence–Discovery Separation . . . . .                      | 8        |
| 2.2 Levin Universal Search . . . . .                                  | 8        |
| 2.3 A Decision Version of Levin Program Search . . . . .              | 9        |
| 2.4 Membership in NP . . . . .                                        | 9        |
| 2.5 NP-Hardness . . . . .                                             | 10       |
| 2.6 Interpretation . . . . .                                          | 11       |
| 2.7 Universal Search and the Blindness Principle . . . . .            | 11       |
| 2.8 The Meta-Search Reduction . . . . .                               | 12       |

|          |                                                                                             |           |
|----------|---------------------------------------------------------------------------------------------|-----------|
| <b>3</b> | <b>Complexity Transfer Theory</b>                                                           | <b>12</b> |
| 3.1      | Where Does the Difficulty Go?                                                               | 12        |
| 3.2      | A Quantitative Transfer Law                                                                 | 12        |
| 3.3      | Repeated Reductions and Exponent Growth                                                     | 13        |
| 3.4      | The Water Analogy                                                                           | 14        |
| 3.5      | Example: BLSP to Ordered Search                                                             | 14        |
| 3.6      | Reduction-to-P Barrier                                                                      | 15        |
| 3.7      | Why Stacking Polynomial Algorithms Is Not Enough                                            | 15        |
| 3.8      | NP-Native Polynomial Computation                                                            | 16        |
| 3.9      | Connection to Blind Witness Search                                                          | 16        |
| 3.10     | Connection to Levin Program Search                                                          | 17        |
| 3.11     | The No-Escape Criterion                                                                     | 17        |
| 3.12     | Third Principle of the Construction                                                         | 18        |
| <b>4</b> | <b>The NP-Native Construction Constraint</b>                                                | <b>18</b> |
| 4.1      | Why the Architecture Must Be Delimited                                                      | 18        |
| 4.2      | External-P Architecture                                                                     | 19        |
| 4.3      | The Native Alternative                                                                      | 19        |
| 4.4      | Native State Space                                                                          | 20        |
| 4.5      | The Crucial Distinction                                                                     | 20        |
| 4.6      | An “NP Algorithm” in the Present Sense                                                      | 21        |
| 4.7      | The Non-Escape Rule                                                                         | 21        |
| 4.8      | Primitive Operations and Decisive Operations                                                | 22        |
| 4.9      | Native Polynomialization                                                                    | 23        |
| 4.10     | Closure Inside the Verification Relation                                                    | 23        |
| 4.11     | Why This Is Not the Same as an Ordinary Reduction                                           | 24        |
| 4.12     | The Delimitation Condition                                                                  | 24        |
| 4.13     | Target Form of the Core Algorithm                                                           | 25        |
| <b>5</b> | <b>The <math>\Phi</math> Model and the Search for a Global <math>\Omega</math> Property</b> | <b>25</b> |
| 5.1      | One Exceptional NP-Complete Problem Is Sufficient                                           | 25        |
| 5.2      | The Idealized $\Phi$ Model                                                                  | 26        |
| 5.3      | Local Structure Is Not the Same as Global Direction                                         | 26        |
| 5.4      | Why a Global Property Is Needed                                                             | 27        |
| 5.5      | The $\Omega$ Property                                                                       | 27        |
| 5.6      | Requirements for a Useful $\Omega$                                                          | 28        |
| 5.7      | Breaking $\Phi$                                                                             | 29        |
| 5.8      | Local Patterns as Evidence of Non-Random Geometry                                           | 29        |
| 5.9      | The Exceptional-Structure Hypothesis                                                        | 30        |
| 5.10     | A Strong Form of the $\Omega$ Criterion                                                     | 30        |
| 5.11     | A Weaker Form                                                                               | 30        |
| 5.12     | The $\Phi$ -to- $\Omega$ Program                                                            | 31        |
| <b>6</b> | <b>Meta-Binary Search and Recursive Self-Oracularization</b>                                | <b>31</b> |
| 6.1      | The Missing Oracle                                                                          | 31        |
| 6.2      | Binary Search Without a Directional Oracle                                                  | 32        |
| 6.3      | Meta-Binary Search                                                                          | 32        |
| 6.4      | Self-Oracularization                                                                        | 33        |
| 6.5      | The Infinite Oracle Tower                                                                   | 33        |
| 6.6      | Holographic Encoding                                                                        | 34        |
| 6.7      | Fixed-Point Formulation                                                                     | 35        |
| 6.8      | Polynomial Compression Requirement                                                          | 35        |

|          |                                                                                                           |           |
|----------|-----------------------------------------------------------------------------------------------------------|-----------|
| 6.9      | Toy Example: Meta-Binary Search                                                                           | 36        |
| 6.10     | From Meta-Binary Search to SAT                                                                            | 36        |
| 6.11     | SAT Self-Oracularization                                                                                  | 37        |
| 6.12     | Holographic SAT State                                                                                     | 38        |
| 6.13     | The Meta-SAT Principle                                                                                    | 38        |
| 6.14     | Connection to the $\Omega$ Property                                                                       | 39        |
| 6.15     | Why Infinite Recursion Does Not Automatically Imply Infinite Size                                         | 39        |
| 6.16     | The Compression Lemma Target                                                                              | 40        |
| 6.17     | The Computation Lemma Target                                                                              | 41        |
| 6.18     | No-Oracle Closure                                                                                         | 41        |
| 6.19     | Section Target                                                                                            | 41        |
| <b>7</b> | <b>Iterated Function Systems as a Finite Model of Infinite Self-Oracularization</b>                       | <b>42</b> |
| 7.1      | Why Iterated Function Systems Appear Naturally                                                            | 42        |
| 7.2      | Finite Rule, Infinite Structure                                                                           | 42        |
| 7.3      | A Binary IFS on Cantor Space                                                                              | 43        |
| 7.4      | From the Assignment Tree to a SAT-Conditioned IFS                                                         | 44        |
| 7.5      | Identification of $\Omega$                                                                                | 45        |
| 7.6      | Recursive Oracle Queries as IFS Addresses                                                                 | 45        |
| 7.7      | The Oracle Attractor                                                                                      | 46        |
| 7.8      | Self-Oracularization as a Fractal Fixed Point                                                             | 46        |
| 7.9      | The Holographic Interpretation                                                                            | 47        |
| 7.10     | Description Complexity                                                                                    | 47        |
| 7.11     | Why Compression Alone Is Not Enough                                                                       | 48        |
| 7.12     | IFS Compression Lemma Target                                                                              | 48        |
| 7.13     | Contractivity Lemma Target                                                                                | 49        |
| 7.14     | Directional Decoding Lemma Target                                                                         | 49        |
| 7.15     | The Complete IFS Route                                                                                    | 50        |
| 7.16     | The Seventh Principle                                                                                     | 50        |
| <b>8</b> | <b>Fourier Holography and Global Spectral Closure</b>                                                     | <b>50</b> |
| 8.1      | Why a Second Holographic Layer Is Needed                                                                  | 50        |
| 8.2      | Fourier Representation of Boolean State Space                                                             | 51        |
| 8.3      | The SAT Landscape as a Signal                                                                             | 51        |
| 8.4      | Fourier Holographic State                                                                                 | 52        |
| 8.5      | Spectralization of the IFS                                                                                | 52        |
| 8.6      | Affine IFS Example                                                                                        | 53        |
| 8.7      | Boolean Fourier Version                                                                                   | 54        |
| 8.8      | Spectral Closure                                                                                          | 54        |
| 8.9      | The Holographic Closure Equation                                                                          | 55        |
| 8.10     | Global Information from Distributed Coefficients                                                          | 55        |
| 8.11     | Interference as Global Aggregation                                                                        | 56        |
| 8.12     | Fourier-Holographic Direction                                                                             | 56        |
| 8.13     | Polynomial Fourier Hologram Target                                                                        | 56        |
| 8.14     | Spectral Fixed-Point Extraction Target                                                                    | 57        |
| 8.15     | Precision Separation Requirement                                                                          | 57        |
| 8.16     | IFS–Fourier Dual Closure                                                                                  | 57        |
| 8.17     | Relationship to the $\Phi$ -Model                                                                         | 58        |
| 8.18     | The Eighth Principle                                                                                      | 58        |
| <b>9</b> | <b>The Concrete <math>\Omega</math> Engine: Harmonic Self-Oracles, Weighted IFS, and Fourier Transfer</b> | <b>59</b> |

|           |                                                                                  |           |
|-----------|----------------------------------------------------------------------------------|-----------|
| 9.1       | Choice of Native NP-Complete Language                                            | 59        |
| 9.2       | Clause Violation Polynomials                                                     | 60        |
| 9.3       | The Leaf Satisfaction Signal                                                     | 60        |
| 9.4       | The Exact Harmonic Oracle Field                                                  | 61        |
| 9.5       | Exact Direction from the Harmonic Field                                          | 61        |
| 9.6       | Meta-Binary Search Reappears                                                     | 62        |
| 9.7       | Fourier Identity for the Harmonic Field                                          | 62        |
| 9.8       | Clause Factors in the Walsh Domain                                               | 63        |
| 9.9       | Local Spectral Transfer Operators                                                | 63        |
| 9.10      | The Fourier Hologram Circuit                                                     | 64        |
| 9.11      | Weighted IFS Representation of Spectral Expansion                                | 64        |
| 9.12      | Contractive Address Maps                                                         | 65        |
| 9.13      | The Weighted Spectral IFS                                                        | 65        |
| 9.14      | Directional Query Functional                                                     | 66        |
| 9.15      | Backward Holographic Evaluation                                                  | 66        |
| 9.16      | Holographic Quotient States                                                      | 66        |
| 9.17      | The $\Omega$ State                                                               | 67        |
| 9.18      | The Central Compression Condition                                                | 67        |
| 9.19      | No Hidden Precision Cost                                                         | 68        |
| 9.20      | The Concrete Direction Algorithm                                                 | 68        |
| 9.21      | Conditional Complexity Bound                                                     | 68        |
| 9.22      | Why the Construction Is NP-Native                                                | 70        |
| 9.23      | Why the Construction Is Holographic                                              | 70        |
| 9.24      | The Engineering Bottleneck                                                       | 70        |
| 9.25      | Ninth Principle: Harmonic-Spectral Self-Oracle                                   | 71        |
| <b>10</b> | <b>Reflective Closure: Using the Construction to Certify Its Own Compression</b> | <b>71</b> |
| 10.1      | The Recursive Proof Idea                                                         | 71        |
| 10.2      | Computation and Proof as Parallel State Spaces                                   | 72        |
| 10.3      | The Reflective Update Operator                                                   | 72        |
| 10.4      | Self-Application                                                                 | 73        |
| 10.5      | The Non-Circularity Condition                                                    | 73        |
| 10.6      | Certificate for the Quotient Lemma                                               | 74        |
| 10.7      | Recursive Certification of Equivalence                                           | 74        |
| 10.8      | Proof IFS                                                                        | 75        |
| 10.9      | Fourier Representation of the Proof State                                        | 75        |
| 10.10     | The Self-Certifying Quotient Condition                                           | 76        |
| 10.11     | Contradiction Strategy Inside the Reflective System                              | 76        |
| 10.12     | Recursive Compression of Counterexamples                                         | 77        |
| 10.13     | Reflective Fixed Point                                                           | 78        |
| 10.14     | Primitive Proof Base                                                             | 78        |
| 10.15     | Reflective Soundness Target                                                      | 79        |
| 10.16     | The Recursive Meme as a Mathematical Architecture                                | 79        |
| 10.17     | Tenth Principle: Reflective Holographic Closure                                  | 79        |
| <b>11</b> | <b>Executable Google Colab Implementation</b>                                    | <b>80</b> |
| 11.1      | Exact executable quotient                                                        | 80        |
| 11.2      | One-Cell Google Colab Program                                                    | 80        |
| 11.3      | Output contract                                                                  | 88        |
| <b>12</b> | <b>Conclusion</b>                                                                | <b>89</b> |
| 12.1      | Holographic closure theorem                                                      | 89        |

# Roadmap

The argument is organized as a sequence of increasingly concrete constructions. Sections 1–4 formulate the directional-information viewpoint and the NP-native design constraint. Section 5 introduces the global property  $\Omega$ . Sections 6–8 develop recursive self-oracularization, IFS closure, and Fourier holography. Section 9 gives the concrete harmonic-spectral engine, Section 10 introduces reflective certification, and Section 11 provides executable code.

## 1 NP-Complete Problems as Blind Witness Search

### 1.1 Motivating Perspective

The standard definition of NP-completeness is formulated in terms of polynomial-time verification and polynomial-time reductions [1, 2]. We use the standard formulation of the **P** versus **NP** Millennium Prize Problem given in Cook’s official Clay Mathematics Institute problem description [8]. In this work, we introduce a *conceptual viewpoint* intended to isolate the source of difficulty that will be used throughout the construction.

The central intuition is simple: some search problems are *guided*, whereas others are *blind*. A guided search receives directional information after each partial choice. A blind search may efficiently verify a complete candidate, but receives no immediate indication of how an incorrect candidate should be modified in order to approach a valid one.

A canonical example of guided search is binary search. Suppose that an unknown integer  $z$  lies in an ordered interval

$$\{0, 1, \dots, N - 1\}.$$

If an oracle answers whether a query  $q$  satisfies  $q < z$ ,  $q = z$ , or  $q > z$ , then every query reveals which half of the remaining search space may be discarded. The search is therefore not merely a sequence of guesses: it is a sequence of guesses equipped with *directional information*. The number of required queries is

$$O(\log N).$$

By contrast, suppose that the only available oracle answers

$$\text{Eq}_z(q) = \begin{cases} 1, & q = z, \\ 0, & q \neq z. \end{cases}$$

A negative answer does not indicate whether  $q$  is too large, too small, or otherwise structurally close to  $z$ . The same search space has become *blind*.

### 1.2 Polynomial Verification Relations

Let  $L \in \mathbf{NP}$ . Then there exists a polynomial  $p$  and a polynomial-time verifier

$$V(x, w) \in \{0, 1\}$$

such that

$$x \in L \iff \exists w \in \{0, 1\}^{p(|x|)} \text{ such that } V(x, w) = 1.$$

For a fixed instance  $x$ , define the acceptance function

$$A_x(w) := V(x, w).$$

The ordinary NP verifier therefore induces a Boolean landscape over the witness space

$$\{0, 1\}^m, \quad m = p(|x|),$$

whose accepting points are precisely the valid witnesses.

The verifier answers the question

“Is this complete candidate a valid witness?”

but, in general, the single bit  $A_x(w) = 0$  does not by itself answer

“Which part of  $w$  should be changed?”

or

“In which direction is a valid witness located?”

This motivates the following model.

### 1.3 Blind Witness Search

**Definition 1.1** (Blind Witness Search). Let  $V(x, w)$  be a polynomial-time verifier and let  $m = p(|x|)$  be the witness length. The *blind witness search problem* associated with  $V$  is the task of finding

$$w^* \in \{0, 1\}^m$$

such that

$$V(x, w^*) = 1,$$

when the search procedure receives only the accept/reject information generated by complete candidate witnesses and no additional directional oracle.

The word *blind* does not mean that the instance  $x$  is hidden. Rather, it refers specifically to the absence of a guaranteed *directional signal in witness space*. The algorithm may inspect the entire input instance and exploit any available structure; what is absent is an oracle that directly identifies which branch of the witness space still contains a solution.

### 1.4 Guided Witness Search

To make the contrast precise, define a prefix-extension oracle.

**Definition 1.2** (Prefix-Extension Oracle). For a fixed instance  $x$ , let

$$E_x(u) = 1$$

if and only if the bit string  $u$  is a prefix of at least one valid witness, that is,

$$E_x(u) = 1 \iff \exists v \in \{0, 1\}^{m-|u|} \text{ such that } V(x, uv) = 1.$$

The oracle  $E_x$  converts blind search into guided search.

**Proposition 1.3** (Witness Recovery from Directional Information). *Assume access to  $E_x$ , and suppose that at least one valid witness exists. Then a valid witness can be recovered using at most  $m$  adaptive extension queries.*

*Proof.* Begin with the empty prefix  $u = \varepsilon$ . At each position, query  $E_x(u0)$ . If

$$E_x(u0) = 1,$$

replace  $u$  by  $u0$ . Otherwise replace  $u$  by  $u1$ . Since the current prefix is maintained so that it extends to at least one valid witness, one of the two extensions must remain valid. Repeating this process for all  $m$  positions produces a string  $w \in \{0, 1\}^m$  satisfying

$$V(x, w) = 1.$$

Therefore the witness is recovered in at most  $m$  guided steps.  $\square$

This proposition captures the essential analogy with binary search: polynomially many decisions suffice once every decision reveals which region of the remaining search space still contains a solution.

## 1.5 The Blindness Gap

The distinction may be summarized as follows:

|                                                          |        |                                                          |
|----------------------------------------------------------|--------|----------------------------------------------------------|
| Verification oracle:<br>$w \mapsto V(x, w) \in \{0, 1\}$ | versus | Directional oracle:<br>$u \mapsto E_x(u) \in \{0, 1\}$ . |
|----------------------------------------------------------|--------|----------------------------------------------------------|

The first oracle recognizes a successful endpoint. The second oracle guides the construction of that endpoint.

In a purely black-box setting, the distinction is exponential. If  $A_x : \{0, 1\}^m \rightarrow \{0, 1\}$  is treated as an arbitrary Boolean oracle having a unique accepting point, then a deterministic algorithm may require  $2^m$  queries in the worst case to identify that point: every rejected candidate eliminates only itself. With the prefix-extension oracle, the same witness can be recovered in  $m$  steps.

Thus the exponential difficulty of blind search can be viewed as the cost of the missing directional information.

## 1.6 NP-Complete Problems from the Blind-Search Perspective

We therefore adopt the following perspective throughout this work:

An NP-complete problem may be viewed as a polynomially verifiable search problem for which no general polynomial-time directional mechanism is known.

This sentence is not intended to replace the standard formal definition of NP-completeness. Rather, it identifies the structural feature targeted by the construction developed in the later sections.

For SAT, for example, a truth assignment

$$a \in \{0, 1\}^n$$

can be checked in polynomial time. If the assignment fails, ordinary verification establishes only that this particular assignment does not satisfy the formula. A hypothetical directional mechanism would instead extract enough information from the formula and the current partial assignment to determine which region of assignment space can still contain a satisfying assignment.

The central objective of the present approach is therefore not to accelerate brute-force verification. It is to remove the blindness.

## 1.7 The Directionalization Principle

This motivates the first principle of the construction.

**Principle 1.4** (Directionalization Principle). *Given a polynomial-time verifier  $V(x, w)$ , seek a polynomial-time computable signal*

$$D(x, u)$$

*defined on partial witnesses  $u$ , such that  $D$  contains sufficient information to select an extension of  $u$  that preserves the existence of a valid completion whenever such a completion exists.*

If such a signal can be computed in polynomial time for an NP-complete problem, and if it can be iterated over only polynomially many witness positions, then blind witness search is transformed into guided witness construction.

The remainder of this work develops a candidate mechanism for performing precisely this transformation.

## 2 Levin Universal Search as an NP-Complete Meta-Search Problem

### 2.1 The Existence–Discovery Separation

The second structural principle of this work concerns a distinction between the *existence* of an algorithm and the *discovery* of that algorithm.

For any fixed finite SAT instance  $F$ , there exists a finite program that correctly resolves that instance. Indeed, because the instance is fixed, a program may simply hard-code either a satisfying assignment, when one exists, or the answer UNSAT, when the formula is unsatisfiable.

Consequently, the fundamental difficulty for a specific instance is not the abstract existence of some correct finite program. The difficulty is to *identify* a useful program without already knowing the answer that the program is intended to compute.

This observation is closely related to Levin’s universal search [3]. Universal search systematically enumerates candidate programs, allocates computational resources among them, and verifies candidate outputs whenever verification is efficient.

Thus, whenever a short and fast correct algorithm exists, universal search will eventually discover and execute it, up to the overhead imposed by its universal scheduling procedure.

### 2.2 Levin Universal Search

Let  $U(p, x)$  denote a fixed universal machine executing program  $p$  on input  $x$ . Informally, Levin search considers programs  $p$  in increasing description length and simulates them with a resource allocation weighted approximately by

$$2^{-|p|}.$$

If a candidate program outputs a witness  $w$ , a polynomial-time verifier tests whether the witness is valid.

For SAT, the verification predicate is

$$V_{\text{SAT}}(F, a) = 1 \iff a \models F,$$

where  $F$  is a Boolean formula and  $a$  is a truth assignment.

If there exists a program  $p^*$  that outputs a satisfying assignment in time  $T$ , then universal search eventually simulates  $p^*$  for enough steps to reproduce that output. The search cost is controlled by both the runtime  $T$  and the description length  $|p^*|$ .

The crucial point is that the universal search mechanism does not initially know which candidate program is the useful one.

### 2.3 A Decision Version of Levin Program Search

Universal search itself is an algorithmic procedure rather than a decision language. To study its complexity in the language of NP-completeness, we introduce the following bounded decision problem.

**Definition 2.1** (Bounded Levin–SAT Program Existence). An instance of BLSP consists of

$$(F, k, t),$$

where  $F$  is a Boolean formula,  $k$  is a program-length bound, and  $t$  is a computation-time bound. The question is whether there exists a program  $p$  satisfying

$$|p| \leq k$$

such that the universal machine  $U(p, F)$  halts within at most  $t$  steps and outputs an assignment  $a$  satisfying

$$a \models F.$$

The bounds  $k$  and  $t$  are represented so that the permitted simulation is polynomially bounded in the total input size; equivalently, one may take  $t$  in unary or restrict attention to a polynomially bounded  $t$ .

The problem asks a meta-level question:

Does there exist a sufficiently short and sufficiently fast program  
that solves this SAT instance?

This formalizes the intuition that the useful solver may already exist inside the universal program space, while the computational task is to locate it.

### 2.4 Membership in NP

**Theorem 2.2.** *BLSP belongs to NP.*

*Proof.* A certificate consists of a candidate program  $p$ .

The verifier first checks that

$$|p| \leq k.$$

It then simulates

$$U(p, F)$$

for at most  $t$  steps. If the program halts and outputs an assignment  $a$ , the verifier checks in polynomial time whether

$$a \models F.$$

Because the simulation is bounded by  $t$ , and  $t$  is polynomially bounded by the encoded input size, the entire verification procedure runs in polynomial time. Therefore

$$\text{BLSP} \in \mathbf{NP}.$$

□

This produces the characteristic asymmetry:

finding the successful program may be difficult,  
while checking a proposed successful program is easy.

## 2.5 NP-Hardness

We now show that the bounded Levin program-existence problem is at least as hard as SAT.

**Theorem 2.3.** *BLSP is NP-hard.*

*Proof.* We give a polynomial-time many-one reduction from SAT.

Let  $F$  be an arbitrary Boolean formula over  $n$  variables. Construct the BLSP instance

$$(F, k(n), t(n)),$$

where

$$k(n) = n + c$$

for a machine-dependent constant  $c$ , and

$$t(n) = q(n)$$

for a polynomial  $q$  sufficiently large to allow a program to print an  $n$ -bit assignment.

If  $F$  is satisfiable, let

$$a \in \{0, 1\}^n$$

be any satisfying assignment. There exists a program  $p_a$  consisting of a fixed output routine together with the hard-coded string  $a$ . Hence

$$|p_a| \leq n + c = k(n),$$

and  $p_a$  outputs  $a$  within  $q(n)$  steps. Therefore

$$(F, k(n), t(n)) \in \text{BLSP}.$$

Conversely, suppose

$$(F, k(n), t(n)) \in \text{BLSP}.$$

Then some program  $p$  outputs within the allowed time an assignment  $a$  satisfying

$$a \models F.$$

Therefore  $F$  is satisfiable.

Thus

$$F \in \text{SAT} \iff (F, k(n), t(n)) \in \text{BLSP}.$$

The transformation is polynomial-time computable, so

$$\text{SAT} \leq_p \text{BLSP}.$$

Hence BLSP is NP-hard.

□

Combining Theorems 2.2 and 2.3, we obtain the following.

**Corollary 2.4.** *BLSP is NP-complete.*

## 2.6 Interpretation

The preceding construction gives a precise version of the following idea:

For a satisfiable SAT instance, a successful program already exists in the space searched by a universal algorithm. The computational difficulty is to identify a successful program without already possessing the satisfying assignment encoded by that program.

The NP-completeness does not arise merely because universal search can execute a polynomial-time SAT solver if one is supplied. Rather, it arises because the *existence of a short program producing a valid witness* can itself encode the SAT decision problem.

This distinction is essential. Executing a known polynomial-time solver is easy. Determining which program in a universal search space is the successful one may encode exactly the original NP-complete search.

## 2.7 Universal Search and the Blindness Principle

The previous section characterized NP search as a form of blindness in witness space. The present section reveals a second layer of blindness: *program-space blindness*.

There are therefore two related search spaces:

$$\underbrace{\{0, 1\}^m}_{\text{witness space}} \quad \text{and} \quad \underbrace{\{0, 1\}^*}_{\text{program space}} .$$

In witness space, the problem is to find

$$w^* \quad \text{such that} \quad V(x, w^*) = 1.$$

In program space, the problem is to find

$$p^* \quad \text{such that} \quad U(p^*, x)$$

produces such a witness efficiently.

A universal search procedure removes the need to know *whether* such a program can in principle be represented: all finite programs are eventually considered. What it does not automatically remove is the cost of locating the correct program.

This yields the second principle of the present work.

**Principle 2.5** (Algorithm Existence versus Algorithm Discovery). *For a fixed finite NP instance, the existence of a finite correct program is not the central obstacle. The obstacle is the efficient discovery of a program whose behavior yields a valid witness without encoding the unknown solution as prior information.*

## 2.8 The Meta-Search Reduction

The relation between the two search layers can be written schematically as

$$\boxed{\text{SAT witness search} \longrightarrow \text{program search} \longrightarrow \text{universal verification}}.$$

A satisfying assignment may be embedded into a short program that prints it. Consequently, searching for such a program can be at least as hard as searching for the witness itself.

The research objective is therefore sharpened:

Do not merely enumerate candidate algorithms;  
construct directional information in program space as well.

If the blindness identified in witness space and the blindness identified in program space can both be removed by a polynomial-time mechanism, then universal search ceases to be a brute-force meta-procedure and becomes a candidate route toward efficient solver synthesis.

## 3 Complexity Transfer Theory

### 3.1 Where Does the Difficulty Go?

A recurring phenomenon in attempts to transform an NP-complete problem into a tractable one is that the computational difficulty does not disappear when the target problem is easy. Instead, the cost may be transferred into the transformation itself.

This motivates the following principle.

**Principle 3.1** (Complexity Transfer Principle). *Suppose a problem  $A$  is transformed into a problem  $B$ , solved there, and then decoded back into a solution of  $A$ . The total complexity is the cost of the entire pipeline,*

$$A \xrightarrow{R} B \xrightarrow{S_B} \text{solution of } B \xrightarrow{D} \text{solution of } A,$$

*not merely the complexity of the solver  $S_B$  for the target problem.*

Thus, reducing an NP-complete problem to an apparently simple polynomial-time problem does not by itself remove the original difficulty. If the reduction must compute information equivalent to the missing NP witness, then the difficulty has merely moved from the solver to the reduction.

We refer to this displacement as *complexity transfer*.

### 3.2 A Quantitative Transfer Law

Let an input  $x$  of size  $n$  be transformed by a reduction  $R$  into an instance  $y = R(x)$  of size

$$|y| = O(n^b).$$

Assume that the reduction itself takes

$$T_R(n) = O(n^a)$$

time, the target problem is solved in

$$T_B(m) = O(m^c)$$

time on inputs of length  $m$ , and reconstruction takes

$$T_D(n) = O(n^d).$$

Then the complete algorithm has running time

$$\begin{aligned} T_A(n) &= T_R(n) + T_B(|R(x)|) + T_D(n) \\ &= O(n^a) + O((n^b)^c) + O(n^d) \\ &= O(n^{\max\{a, bc, d\}}). \end{aligned}$$

This gives the first form of the transfer law:

$$\boxed{\deg(A) = \max\{a, bc, d\}.$$

Even when  $B$  has an extremely efficient algorithm, a large exponent in the reduction or a large size blow-up can dominate the final complexity.

### 3.3 Repeated Reductions and Exponent Growth

The effect becomes more visible when many polynomial transformations are composed.

Suppose

$$A_0 \xrightarrow{R_1} A_1 \xrightarrow{R_2} A_2 \xrightarrow{R_3} \dots \xrightarrow{R_k} A_k,$$

and reduction  $R_i$  maps an instance of size  $m$  to one of size

$$O(m^{b_i}).$$

After  $k$  reductions, the final instance may have size

$$O(n^{b_1 b_2 \dots b_k}).$$

If the final algorithm runs in degree  $c$ , then its contribution becomes

$$O(n^{c b_1 b_2 \dots b_k}).$$

Consequently, a sequence of individually polynomial constructions may produce a polynomial with a very large degree. Bounds resembling

$$O(n^{21})$$

can therefore arise naturally from repeated representation expansion, simulation overhead, auxiliary data structures, normalization steps, and nested polynomial subroutines.

The important point is not the specific exponent 21. The structural point is that

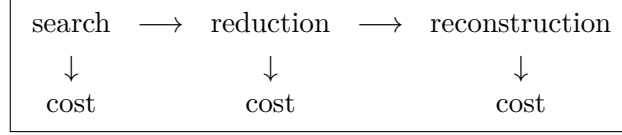
$$\boxed{\text{polynomial} \circ \text{polynomial} \circ \dots \text{ remains polynomial, but its degree may grow dramatically.}}$$

This phenomenon will be called *polynomial complexity accumulation*.

### 3.4 The Water Analogy

Complexity transfer may be visualized as a conservation-like phenomenon.

Imagine computational difficulty as water flowing through a sequence of containers. Replacing one container by an easier one does not necessarily remove the water. Unless an actual structural simplification occurs, the water reappears elsewhere:



One may reduce the complexity of the central solver while increasing the complexity of the transformation feeding that solver.

For this reason, the objective is not merely to relocate the difficult step. The objective is to destroy the need for that step.

### 3.5 Example: BLSP to Ordered Search

Consider the NP-complete problem BLSP from Section 2. Suppose one attempts to transform a BLSP instance into an ordered list so that binary search may locate the desired program.

Binary search itself requires only

$$O(\log N)$$

queries over a search space of size  $N$ , provided that each query supplies a correct left/right decision.

The essential question is therefore not whether binary search is efficient. It is. The essential question is:

How is the ordering predicate constructed?

Suppose a predicate

$$G(x, q) \in \{-1, 0, +1\}$$

is intended to tell the search procedure whether the desired solution lies to the left of  $q$ , equals  $q$ , or lies to the right of  $q$ .

If computing  $G$  requires determining whether some region of program space contains a valid SAT-solving program, then  $G$  may already encode the original NP-complete decision problem.

In that case the search stage has complexity

$$O(\log N),$$

but every search step invokes a difficult directional predicate.

The total cost is then approximately

$$O(T_G(n) \log N),$$

not merely  $O(\log N)$ .

The apparent speed of binary search has therefore not solved the original problem. The complexity has been transferred into  $G$ .

### 3.6 Reduction-to-P Barrier

This observation has an immediate formal consequence.

**Proposition 3.2** (Direct Reduction Barrier). *Let  $A$  be NP-complete and let  $B \in \mathbf{P}$ . If there exists a polynomial-time many-one reduction*

$$A \leq_p B,$$

*then*

$$\mathbf{P} = \mathbf{NP}.$$

*Proof.* Let  $R$  be the polynomial-time reduction from  $A$  to  $B$ , and let  $S_B$  be a polynomial-time decision algorithm for  $B$ . Then

$$x \in A \iff R(x) \in B.$$

Therefore the composition

$$x \mapsto R(x) \mapsto S_B(R(x))$$

decides  $A$  in polynomial time. Since  $A$  is NP-complete, every language in  $\mathbf{NP}$  reduces to  $A$ , hence every language in  $\mathbf{NP}$  belongs to  $\mathbf{P}$ . Since

$$\mathbf{P} \subseteq \mathbf{NP},$$

we conclude

$$\mathbf{P} = \mathbf{NP}.$$

□

Thus a successful polynomial reduction of an NP-complete problem to an ordinary problem such as ordered search is not merely a preprocessing trick. It already contains the essential content of a proof of  $\mathbf{P} = \mathbf{NP}$ .

### 3.7 Why Stacking Polynomial Algorithms Is Not Enough

Suppose an attempted construction uses polynomial-time components

$$A_1, A_2, \dots, A_k.$$

It is tempting to reason that because every component belongs to  $\mathbf{P}$ , their combination should reveal an efficient solution to an NP-complete problem.

However, the relevant question is not whether the components are individually polynomial. It is whether the composition provides the missing information without invoking an operation equivalent to the original NP-complete task.

A construction may contain only polynomial-looking subroutines while still failing for one of three reasons:

1. the number of subroutine invocations is exponential;
2. intermediate representations grow too rapidly;
3. one supposedly simple predicate implicitly asks an NP-complete question.

Thus the important invariant is not

“every box is polynomial,”

but rather

the complete information flow must remain polynomial.

### 3.8 NP-Native Polynomial Computation

We now introduce terminology for the alternative strategy pursued here.

Complexity classes such as **P** and **NP** classify decision problems, not styles of algorithms. Consequently, any deterministic algorithm that decides an NP-complete problem in polynomial time would, by definition, place that problem in **P**.

Nevertheless, there is a useful architectural distinction between two approaches.

**Definition 3.3** (P-Reduction Architecture). A *P-reduction architecture* attempts to solve an NP-complete problem by repeatedly translating it into already tractable polynomial-time primitives.

**Definition 3.4** (NP-Native Polynomial Architecture). An *NP-native polynomial architecture* operates directly on the verification relation

$$V(x, w)$$

or directly on the witness/program space associated with the NP-complete problem, and seeks to construct the missing witness using only a polynomial number of polynomial-time transformations, without first assuming an efficient reduction to an already ordered or guided search problem.

The phrase *NP-native* describes where the algorithm is designed to operate, not a new complexity class.

If such an architecture deterministically solves an NP-complete problem in polynomial time, then the final consequence is of course

$$\mathbf{P} = \mathbf{NP}.$$

The conceptual difference is methodological:

Do not begin by forcing NP into a pre-existing P geometry.  
Instead, construct polynomial guidance inside the NP geometry itself.

### 3.9 Connection to Blind Witness Search

This principle connects directly to Section 1.

The naive route is

$$\text{blind NP search} \longrightarrow \text{reduction} \longrightarrow \text{guided P problem}.$$

Complexity Transfer Theory warns that the missing guidance may simply be hidden inside the reduction.

The proposed route is instead

blind NP search  $\longrightarrow$  internal directionalization  $\longrightarrow$  guided NP-native search.

That is, rather than asking

“To which easy problem can SAT be converted?”,

we ask

“Can SAT generate its own polynomial directional signal?”

This is the central distinction.

### 3.10 Connection to Levin Program Search

The same idea applies to the program space introduced in Section 2.

A universal search enumerates candidate programs blindly. One possible strategy is to repeatedly reduce questions about those programs to simpler problems. Complexity Transfer Theory predicts that the missing information may reappear in the reductions.

The alternative is to define an intrinsic signal

$$D_{\text{prog}}(x, p)$$

that reveals how the current candidate program should be transformed toward a successful one.

Thus the two directionalization problems become

$$D_{\text{wit}}(x, w)$$

for witness space, and

$$D_{\text{prog}}(x, p)$$

for program space.

A successful construction should avoid transferring the difficulty between these spaces.

### 3.11 The No-Escape Criterion

We therefore impose the following criterion on the remainder of the proof.

**Principle 3.5** (No-Escape Criterion). *At every stage of the construction, all complexity introduced by representation changes, reductions, oracle-like predicates, witness reconstruction, program synthesis, and directional signals must be explicitly accounted for.*

*No step is considered a genuine simplification if it merely relocates an NP-complete subproblem into an auxiliary predicate.*

Equivalently, if the final algorithm has the form

$$A = A_k \circ A_{k-1} \circ \cdots \circ A_1,$$

then the proof must establish a uniform polynomial bound

$$T_A(n) \leq n^{O(1)}$$

for the *entire composition*, including every intermediate representation and every call to the directional mechanism.

### 3.12 Third Principle of the Construction

The third principle of this work may now be stated succinctly.

**Principle 3.6** (Native Directionality). *An efficient solution to NP-complete search should not obtain its direction by transferring the original NP-complete difficulty into a preprocessing reduction. Direction should instead be generated intrinsically inside the NP witness or program space with polynomial total cost.*

The goal is therefore not

$$\boxed{\text{NP} \xrightarrow{\text{expensive encoding}} \text{easy P search}}$$

but rather

$$\boxed{\text{NP blind search} \xrightarrow{\text{polynomial intrinsic signal}} \text{NP-native guided search.}}$$

If such intrinsic directionality can be constructed uniformly for an NP-complete problem, then the blindness identified in the preceding sections is removed without allowing the complexity to escape into the reduction.

## 4 The NP-Native Construction Constraint

### 4.1 Why the Architecture Must Be Delimited

The previous section introduced Complexity Transfer Theory: when an NP-complete problem is repeatedly translated into already tractable polynomial-time problems, the missing difficulty may reappear in the translation, ordering predicate, reconstruction map, or intermediate representation.

The present section therefore imposes an architectural restriction on the construction.

The core mechanism should not attempt to solve an NP-complete problem by first converting it into an unrelated problem already known to belong to **P**, solving that external problem, and then decoding its answer. Instead, the mechanism should operate directly on the native structure of the NP problem itself.

Informally:

$$\boxed{\text{Do not solve NP by escaping from NP.}}$$

Rather, the goal is

$$\boxed{\text{operate inside the NP verification structure and make that operation polynomial.}}$$

This distinction is methodological rather than terminological. In standard complexity theory, any deterministic polynomial-time algorithm for an NP-complete language places that language in **P**. Nevertheless, the internal architecture by which such an algorithm is obtained may be very different from a reduction to an already known problem in **P**.

## 4.2 External-P Architecture

We first formalize the architecture that the present approach seeks to avoid.

**Definition 4.1** (External-P Architecture). Let  $L$  be an NP-complete language. An *external-P architecture* is a pipeline of the form

$$x \xrightarrow{R} y \xrightarrow{A_P} z \xrightarrow{D} \text{Ans}_L(x),$$

where  $A_P$  is an algorithm designed to solve a problem  $Q \in \mathbf{P}$ , while  $R$  translates the original NP-complete instance into an instance of  $Q$ , and  $D$  reconstructs the answer to  $L$ .

Such an architecture is not intrinsically invalid. Indeed, if  $R$  and  $D$  are polynomial and the reduction is correct, then it would yield a polynomial-time algorithm for  $L$ .

The concern is instead structural: if the transformation  $R$  must already discover the missing witness geometry, then the tractability of  $A_P$  has not explained the tractability of the original problem.

The apparent simplification may have the form

$$\underbrace{\text{hard NP structure}}_{\text{input}} \xrightarrow{\text{hard translation}} \underbrace{\text{easy P structure}}_{\text{target}}.$$

The easy target problem then receives information whose expensive discovery has already occurred.

## 4.3 The Native Alternative

We therefore define a different architecture.

**Definition 4.2** (NP-Native Operator). Let  $L \in \mathbf{NP}$  be represented by a polynomial-time verification relation

$$V(x, w).$$

An *NP-native operator* is a transformation whose primary objects are drawn directly from the verification structure of  $L$ , such as

$$x, \quad w, \quad u, \quad V(x, w), \quad \mathcal{W}_x,$$

where

$$\mathcal{W}_x := \{w : V(x, w) = 1\},$$

or from representations canonically derived from these objects, and whose purpose is to transform, restrict, organize, compare, or navigate the witness space without first replacing the task by an unrelated language already known to be in  $\mathbf{P}$ .

The phrase *NP-native* does not denote a new classical complexity class. It denotes the location at which the mechanism acts.

An NP-native operator may use ordinary polynomial-time arithmetic, Boolean operations, indexing, sorting of explicitly available polynomial-size data, or other elementary subroutines. What it must not do is derive its decisive power by silently outsourcing the missing NP search to an auxiliary NP-complete predicate disguised as preprocessing.

#### 4.4 Native State Space

For a fixed input  $x$ , let the candidate-witness space be

$$\Omega_x := \{0, 1\}^{p(|x|)}.$$

The standard verifier partitions this space into

$$\Omega_x = \mathcal{W}_x \cup \mathcal{N}_x,$$

where

$$\mathcal{W}_x = \{w : V(x, w) = 1\}$$

is the accepting region and

$$\mathcal{N}_x = \{w : V(x, w) = 0\}$$

is the rejecting region.

An external-P strategy may attempt to map  $\Omega_x$  into a new geometry whose navigation is already easy.

The NP-native strategy instead asks whether one can define operations

$$\Phi_x : \Omega_x \rightarrow \Omega_x,$$

or, more generally,

$$\Phi_x : \mathcal{S}_x \rightarrow \mathcal{S}_x,$$

where  $\mathcal{S}_x$  is a polynomially representable state space derived from the witness relation, such that repeated application of  $\Phi_x$  reveals a valid witness in polynomially many steps.

The desired architecture is therefore

$$s_0 \xrightarrow{\Phi_x} s_1 \xrightarrow{\Phi_x} s_2 \xrightarrow{\Phi_x} \cdots \xrightarrow{\Phi_x} s_T,$$

with

$$T \leq |x|^{O(1)}$$

and with  $s_T$  determining a witness  $w^*$  satisfying

$$V(x, w^*) = 1$$

whenever such a witness exists.

#### 4.5 The Crucial Distinction

The distinction may be summarized as follows.

The rejected architecture is

|                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------|
| NP instance $\longrightarrow$ representation of a known P problem $\longrightarrow$ P solver $\longrightarrow$ NP answer. |
|---------------------------------------------------------------------------------------------------------------------------|

The target architecture is

$$\boxed{\begin{array}{l} \text{NP instance} \longrightarrow \text{NP-native transformation} \\ \longrightarrow \text{NP-native transformation} \longrightarrow \cdots \longrightarrow \text{witness.} \end{array}}$$

If the second pipeline runs in polynomial time, then the final algorithm is, of course, a polynomial-time algorithm in the ordinary complexity-theoretic sense.

What differs is the source of the tractability.

The first architecture attempts to inherit tractability from a different problem.

The second architecture attempts to *create tractability internally*.

#### 4.6 An “NP Algorithm” in the Present Sense

The phrase

“an NP algorithm that solves NP”

will be used in this work only as shorthand for an NP-native mechanism.

It does *not* mean that there exists a separate standard class of “NP algorithms” distinct from nondeterministic polynomial-time machines. Instead, it means that the construction is formulated directly in terms of the same witness relation responsible for membership in **NP**.

The mechanism is allowed to exploit facts of the form

$$\exists w \, V(x, w) = 1,$$

the internal structure of candidate witnesses, relations among partial witnesses, verifier responses, clause structure, constraint structure, or program-witness structure.

Its job is to convert this native NP structure into deterministic directional information.

Thus the slogan

$$\boxed{\text{NP solves NP in order to become P}}$$

will be understood as the following precise research program:

$$\boxed{\begin{array}{l} \text{Construct a deterministic polynomial-time process} \\ \text{whose states and transitions are defined directly} \\ \text{from an NP verification relation.} \end{array}}$$

If successful for one NP-complete problem, the ordinary consequence is

$$\mathbf{P} = \mathbf{NP}.$$

#### 4.7 The Non-Escape Rule

We now strengthen the No-Escape Criterion of Section 3.

**Principle 4.3** (Non-Escape Rule). *The decisive operation of the construction must not obtain its power by reducing the current NP-complete search state to an independently tractable problem whose representation already contains the missing directional information.*

*Instead, every decisive transition must be definable directly from the native verifier, witness space, constraint system, or program space.*

This rule does not forbid all polynomial-time subroutines. Such a ban would be meaningless, since any eventual polynomial-time algorithm necessarily consists of polynomial-time operations.

The rule forbids a more specific architectural maneuver:

hiding the breakthrough inside the construction of an easy instance.

For example, if SAT is transformed into an ordered array whose median query reveals which half contains a satisfying assignment, then the central question is not whether binary search is allowed. The central question is how the ordering was obtained.

If obtaining that ordering already requires deciding whether a region contains a satisfying assignment, then the decisive NP information entered before binary search began.

The proposed construction seeks the opposite direction: derive the ordering, gradient, orientation, or equivalent signal *from the NP-native structure itself* by a polynomial process.

## 4.8 Primitive Operations and Decisive Operations

To make the restriction precise, we distinguish between two kinds of operations.

**Definition 4.4** (Primitive Polynomial Operation). A *primitive polynomial operation* is an ordinary computational step whose input is explicitly available polynomial-size data and whose output can be computed in deterministic polynomial time without answering a hidden existential question over an exponentially large witness space.

Examples include evaluating the verifier on a specified witness, computing a clause score, updating an explicitly represented state, comparing two explicit integers, multiplying polynomial-size matrices, or sorting an explicitly given polynomial-size list.

**Definition 4.5** (Decisive Operation). A *decisive operation* is a step whose output determines which exponentially large region of witness or program space may be discarded, retained, or preferred.

The proof must show that every decisive operation can itself be implemented using only polynomially many primitive polynomial operations.

This distinction blocks a common circularity. One cannot define

$$D(x, u) = \begin{cases} 0, & \text{if a satisfying completion exists under } u, \\ 1, & \text{otherwise,} \end{cases}$$

and then claim that  $D$  solves SAT efficiently merely because one call to  $D$  returns one bit.

The cost of computing  $D$  is the entire issue.

An acceptable NP-native construction must derive the equivalent directional bit without assuming the existential search it is meant to replace.

## 4.9 Native Polynomialization

We can now state the transformation sought by this paper.

**Definition 4.6** (Native Polynomialization). Let  $L$  be NP-complete with verifier  $V(x, w)$ . A *native polynomialization* of  $L$  is a deterministic process

$$\mathcal{A}_V$$

such that:

1. its state is polynomially representable;
2. its update rule is defined directly from  $x$ ,  $V$ , and polynomially representable structures derived from them;
3. each update is computable in polynomial time;
4. the number of updates is polynomially bounded;
5. no update invokes an auxiliary oracle for an NP-complete language;
6. if  $x \in L$ , the terminal state yields a valid witness;
7. if  $x \notin L$ , the terminal state certifies rejection according to the decision procedure implemented by the construction.

The phrase *polynomialization* emphasizes that the algorithm does not begin by assuming that NP has already been translated into a known P structure. Instead, it attempts to turn the native NP dynamics themselves into a polynomial-time deterministic process.

## 4.10 Closure Inside the Verification Relation

The key object is therefore not an external target language but the verifier relation itself.

Let

$$R_V := \{(x, w) : V(x, w) = 1\}.$$

The construction seeks a family of polynomial-time maps

$$\Phi_1, \Phi_2, \dots, \Phi_r$$

acting on polynomially representable objects derived from  $R_V$ , such that their composition yields enough information to construct a witness:

$$\Phi_r \circ \dots \circ \Phi_2 \circ \Phi_1(x) \mapsto w^*.$$

The objective is to remain inside the semantic universe generated by

$$(x, w, V(x, w)).$$

This motivates the following principle.

**Principle 4.7** (Native Closure Principle). *All nontrivial information used to direct the search should be generated by operations whose inputs remain within a polynomially representable closure of the original NP verification relation.*

In informal language:

The algorithm should learn how to navigate NP from NP itself.

#### 4.11 Why This Is Not the Same as an Ordinary Reduction

At first glance, one may object that any deterministic polynomial algorithm is ultimately a composition of polynomial transformations, so the distinction seems artificial.

The distinction becomes real once the *semantic source* of the directional information is tracked.

In an external-P reduction, the representation is engineered until an already known solver can exploit it.

In an NP-native construction, the representation is engineered only to expose relations already latent in the verifier structure, while the directional mechanism itself must still be derived.

Schematically:

|             |              |                                                  |                      |
|-------------|--------------|--------------------------------------------------|----------------------|
| External-P: | NP structure | $\xrightarrow{\text{encode solution geometry}}$  | known easy geometry, |
| NP-native:  | NP structure | $\xrightarrow{\text{extract internal relation}}$ | direction inside NP. |

The second transformation is useful only if the extracted relation is cheaper to compute than solving the original existential problem directly.

This requirement will be tested explicitly in later sections.

#### 4.12 The Delimitation Condition

We can now state the fourth principle of the construction.

**Principle 4.8** (Delimitation Condition). *The central algorithmic mechanism must not rely on the fact that some external target problem is already known to be in  $\mathbf{P}$ . Its decisive progress must arise from an NP-native transformation acting directly on the verification or witness structure of the NP-complete source problem.*

Equivalently, the intended route is not

$\text{NP} \rightarrow \text{P problem} \rightarrow \text{P algorithm}$

but

$\text{NP} \rightarrow \text{NP-native dynamics} \rightarrow \text{polynomial deterministic behavior.}$

The final state belongs to ordinary  $\mathbf{P}$  if the runtime is polynomial. However, the construction is obtained by polynomializing an NP-native process rather than by importing a pre-existing P solver.

---

**Algorithm 1** Abstract NP-Native Polynomialization

---

**Require:** Instance  $x$  of an NP-complete language and verifier  $V$

1: Construct a polynomial-size native state  $s_0$  from  $x$  and  $V$

2: **for**  $i = 0, 1, \dots, T(|x|) - 1$  **do**

3:     Compute an intrinsic signal

$$d_i \leftarrow D_V(x, s_i)$$

4:     Update the native state

$$s_{i+1} \leftarrow \Phi_V(x, s_i, d_i)$$

5: **end for**

6: Decode the terminal state into a witness or rejection decision

---

### 4.13 Target Form of the Core Algorithm

The desired core algorithm will therefore have the form

The proof burden is then concentrated into three questions:

1. Can  $D_V$  be computed in polynomial time?
2. Does  $\Phi_V$  preserve the existence of a valid solution?
3. Is  $T(|x|)$  polynomially bounded?

This is the precise form of the claim that the algorithm should “remain in NP” while being transformed into a polynomial deterministic procedure.

The next section must therefore identify the native state  $s$ , the intrinsic directional signal  $D_V$ , and the update rule  $\Phi_V$ .

## 5 The $\Phi$ Model and the Search for a Global $\Omega$ Property

### 5.1 One Exceptional NP-Complete Problem Is Sufficient

The preceding sections delimit the architecture of the desired construction. The next question is therefore not whether every NP-complete problem must be attacked simultaneously. It is sufficient to identify one NP-complete language  $L^*$  possessing an exceptional structural property that makes its native search polynomial, by the standard reduction framework for NP-completeness [1, 2].

Indeed, if

$$L^* \text{ is NP-complete}$$

and there exists a deterministic polynomial-time algorithm deciding  $L^*$ , then every language in **NP** reduces to  $L^*$  in polynomial time, and therefore

$$\mathbf{P} = \mathbf{NP}.$$

Consequently, the problem may be reframed as follows:

Find one NP-complete problem whose native structure admits  
a polynomial global navigation law.

Boolean satisfiability will be used as the primary candidate.

## 5.2 The Idealized $\Phi$ Model

To identify what must be broken, we first define an idealized model of maximal blindness.

Let

$$\Omega_n = \{0, 1\}^n$$

be a witness space. Consider a verifier landscape

$$f : \Omega_n \rightarrow \{0, 1\}.$$

The most hostile possible search geometry is one in which, from the viewpoint of every efficiently observable local statistic, the location of accepting points is indistinguishable from an almost unstructured placement.

We denote this idealized regime by  $\Phi$ .

**Definition 5.1** ( $\Phi$ -Blind Search Model). A witness-search problem is said to behave according to the idealized  $\Phi$ -model if polynomially bounded local observations of a candidate state fail to provide a reliable directional signal indicating which macroscopic region of witness space contains an accepting witness.

The  $\Phi$ -model is not introduced as a theorem asserting that  $\mathbf{P} \neq \mathbf{NP}$ . It is a reference model describing what a perfectly blind NP search would look like.

In the limiting intuition, the verifier behaves like a nearly ideal hidden target generator: a wrong witness may be recognizable as wrong, while its failure reveals essentially no useful global information about where a valid witness lies.

Schematically,

$$w \mapsto f(w)$$

reveals

accept or reject

but not

which exponentially large region should be searched next.

If SAT behaved exactly like this idealized object at every useful scale, then the directionalization program of the preceding sections would have no structure to exploit.

## 5.3 Local Structure Is Not the Same as Global Direction

Boolean formulas are not arbitrary black-box functions. Their syntax exposes substantial local combinatorial information.

For a CNF formula

$$F = C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

one may directly inspect, among other objects,

variables,      clauses,      literal occurrences,      local conflicts,

unit implications, clause-variable incidence, partial-assignment effects.

These structures already distinguish SAT from a pure black-box hidden-target problem.

However, local structure alone does not yet solve the problem.

A local statistic may correctly describe the neighborhood of a candidate assignment while failing to identify the global region containing a satisfying assignment. Local signals may conflict, cancel, form cycles, or lead toward local minima.

Thus we distinguish two levels of information.

**Definition 5.2** (Local Property). A property  $L(F, s)$  is *local* if its value is determined by a polynomially bounded neighborhood, bounded-order interaction, or explicitly restricted subset of the current SAT state  $s$ .

**Definition 5.3** (Global Property). A property  $G(F, s)$  is *global* if it summarizes a relation involving the formula as a whole, or a macroscopic partition of its witness space, and its directional meaning cannot be reduced to inspecting only one bounded local neighborhood.

The distinction is conceptual rather than purely syntactic. A global property may still be computed by composing local operations, provided the final quantity captures a global constraint on the witness space.

## 5.4 Why a Global Property Is Needed

The target is not merely a score

$$E_F(w)$$

that says whether the current assignment looks locally better or worse.

The desired object must answer a stronger question:

Which large region of the witness space remains compatible with a solution?

A useful global property must therefore induce a partition or orientation of the native SAT state space.

Let

$$\mathcal{S}_F$$

denote a polynomially representable native state space associated with a formula  $F$ . A candidate global property has the form

$$\Omega(F, s).$$

The symbol  $\Omega$  is used deliberately: it denotes the property intended to break the idealized  $\Phi$ -blindness.

## 5.5 The $\Omega$ Property

**Definition 5.4** (Candidate  $\Omega$  Property). Let  $F$  be a Boolean formula and  $s \in \mathcal{S}_F$  a native SAT state. A candidate  $\Omega$ -property is a polynomially computable global quantity

$$\Omega(F, s)$$

whose value determines a nontrivial macroscopic restriction, orientation, or partition of the remaining witness space.

The essential requirement is that  $\Omega$  contain information not already available from blind verification alone.

If

$$\mathcal{R}(s) \subseteq \{0, 1\}^n$$

denotes the witnesses still represented by state  $s$ , then  $\Omega(F, s)$  should allow construction of a new state  $s'$  satisfying

$$\mathcal{R}(s') \subsetneq \mathcal{R}(s)$$

while preserving at least one satisfying witness whenever the current region contains one:

$$\mathcal{R}(s) \cap \text{SAT}(F) \neq \emptyset \implies \mathcal{R}(s') \cap \text{SAT}(F) \neq \emptyset.$$

Here

$$\text{SAT}(F) := \{w \in \{0, 1\}^n : w \models F\}.$$

This is the central preservation condition.

## 5.6 Requirements for a Useful $\Omega$

For  $\Omega$  to yield a genuine polynomialization rather than another hidden oracle, it must satisfy several properties.

1. **Polynomial computability.** There exists a deterministic polynomial  $q$  such that

$$T_\Omega(F, s) \leq q(|F|).$$

2. **Globality.** The value of  $\Omega$  must constrain a macroscopic subset of the remaining witness space rather than merely rank one neighboring assignment.
3. **Directional power.** From  $\Omega(F, s)$ , one must be able to construct a successor state

$$s' = \Psi(F, s, \Omega(F, s))$$

that represents strict progress.

4. **Solution preservation.** Whenever the current state represents at least one satisfying witness, the chosen successor must continue to represent at least one satisfying witness.
5. **Polynomial convergence.** Repeated application must terminate after at most

$$|F|^{O(1)}$$

updates.

6. **No hidden NP oracle.** Computing  $\Omega$  must not itself require answering a question equivalent to

$$\exists w \, w \models F$$

over an exponentially large unrestricted subspace.

These conditions distinguish a useful  $\Omega$  from an oracle such as

$$\Omega_{\text{trivial}}(F, u) = \begin{cases} 1, & \exists v \text{ such that } uv \models F, \\ 0, & \text{otherwise.} \end{cases}$$

The latter would provide perfect direction, but computing it is precisely the problem to be solved.

## 5.7 Breaking $\Phi$

The role of  $\Omega$  can now be expressed as a symmetry-breaking operation.

Under the idealized  $\Phi$ -model, two enormous regions of witness space may appear computationally indistinguishable with respect to all available local observations.

The desired property satisfies

$$\Omega(F, S_0) \neq \Omega(F, S_1)$$

for two such regions  $S_0, S_1$ , in a manner correlated with preservation of the satisfying set.

Thus the intended transition is

|                                                                                                           |
|-----------------------------------------------------------------------------------------------------------|
| $\Phi\text{-blindness} \xrightarrow{\Omega} \text{global asymmetry} \xrightarrow{\Psi} \text{direction.}$ |
|-----------------------------------------------------------------------------------------------------------|

This is the central conceptual move of the present approach.

## 5.8 Local Patterns as Evidence of Non-Random Geometry

SAT formulas are generated by explicit symbolic constraints. As a result, their witness landscapes need not possess the complete symmetry of the idealized  $\Phi$ -model.

Even before a global law is identified, local structure demonstrates that the landscape is not represented merely by an opaque lookup table.

For example, flipping a single variable modifies exactly those clauses in which the corresponding literal occurs. Partial assignments can activate unit clauses. Clauses sharing variables generate dependency structure. Conflicting assignments generate reusable information about combinations of choices.

These phenomena show that SAT exposes correlations between nearby pieces of the witness representation.

However, the proposed proof requires more than correlations.

It requires a quantity that aggregates them into a globally valid direction.

The intended progression is therefore

|                                                               |
|---------------------------------------------------------------|
| $\text{local pattern} \not\Rightarrow \text{global solution}$ |
|---------------------------------------------------------------|

but potentially

|                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------|
| $\text{all local relations} \xrightarrow{\text{global aggregation}} \Omega \xrightarrow{\text{direction}} \text{polynomial search.}$ |
|--------------------------------------------------------------------------------------------------------------------------------------|

## 5.9 The Exceptional-Structure Hypothesis

We now state the working hypothesis that motivates the next stage.

**Principle 5.5** (Exceptional-Structure Hypothesis). *There exists at least one NP-complete problem  $L^*$  whose native verification structure admits a polynomially computable global property  $\Omega$  that produces solution-preserving directional progress over its witness space.*

For the remainder of the construction, SAT is taken as the candidate  $L^*$ .

The burden is therefore reduced to discovering a concrete  $\Omega$  and proving the required properties.

## 5.10 A Strong Form of the $\Omega$ Criterion

An especially useful form of  $\Omega$  would induce a balanced reduction of the remaining candidate space.

Suppose a state  $s$  represents a set

$$\mathcal{R}(s).$$

If one update produces  $s'$  satisfying

$$|\mathcal{R}(s')| \leq \alpha |\mathcal{R}(s)|$$

for some uniform constant

$$0 < \alpha < 1,$$

while preserving satisfiability, then after  $k$  steps

$$|\mathcal{R}(s_k)| \leq \alpha^k 2^n.$$

Hence

$$k = O(n)$$

iterations suffice to shrink an exponential witness space to constant size.

This would be the direct analogue of binary search, but with a crucial difference:

the directional partition would be generated intrinsically from SAT,  
not supplied by an external ordering oracle.

Balanced shrinkage is stronger than necessary, but it provides a clean target for the construction.

## 5.11 A Weaker Form

The property need not halve the space at every step.

It is enough that a polynomially bounded potential

$$\mu(F, s)$$

decrease monotonically under the  $\Omega$ -guided update:

$$\mu(F, s_{i+1}) < \mu(F, s_i),$$

with

$$0 \leq \mu(F, s) \leq |F|^c$$

for some constant  $c$ .

Then at most

$$|F|^c$$

strict decreases can occur.

This yields an alternative route:

$$\boxed{\Omega \rightarrow \text{global progress measure} \rightarrow \text{polynomial termination.}}$$

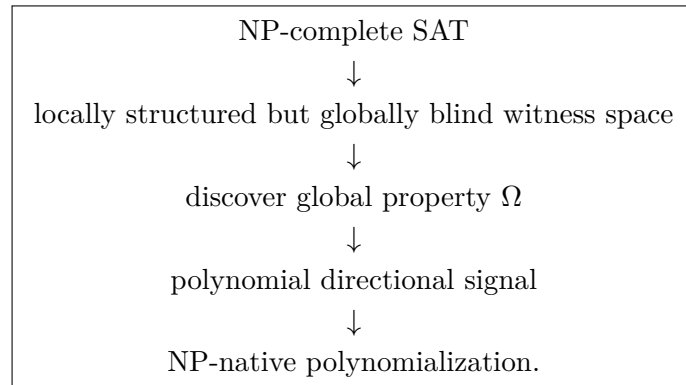
The construction may therefore seek either a balanced-space separator or a global monotone potential.

## 5.12 The $\Phi$ -to- $\Omega$ Program

The fifth principle of this work is now:

**Principle 5.6** ( $\Phi$ -to- $\Omega$  Principle). *Model the maximally difficult NP search as an approximately structureless  $\Phi$ -landscape. Then search for a global invariant  $\Omega$  in a specific NP-complete problem that provably distinguishes solution-preserving directions in polynomial time.*

The intended chain is



The next stage of the proof must therefore stop speaking about  $\Omega$  abstractly and propose an explicit candidate formula for it.

## 6 Meta-Binary Search and Recursive Self-Oracularization

### 6.1 The Missing Oracle

Binary search is efficient because every query returns directional information. Let a totally ordered search space be

$$X = \{0, 1, \dots, N-1\},$$

and let  $z \in X$  be the hidden target. Ordinary binary search assumes access to a directional predicate

$$O_z(q) \in \{-1, 0, +1\},$$

where

$$O_z(q) = \begin{cases} -1, & z < q, \\ 0, & z = q, \\ +1, & z > q. \end{cases}$$

Given  $O_z$ , one query discards approximately half of the remaining search space, yielding the familiar bound

$$O(\log N).$$

The central difficulty is therefore not the bisection rule itself. It is the existence of the directional oracle.

This section asks a different question:

Can the oracle be generated by the search process itself?

This question motivates *meta-binary search*.

## 6.2 Binary Search Without a Directional Oracle

Suppose the target space remains ordered, but the directional oracle is not available. A midpoint query  $q$  no longer answers whether the target lies to the left or right.

The ordinary transition

$$I \longrightarrow I_{\text{left}} \quad \text{or} \quad I_{\text{right}}$$

therefore becomes undefined.

Let

$$\mathcal{B}(I)$$

denote binary search over interval  $I$ , and let

$$\mathcal{D}(I, q)$$

denote the missing decision determining which half of  $I$  should survive.

Ordinary binary search assumes

$$\mathcal{D}$$

as an external primitive.

Meta-binary search instead promotes  $\mathcal{D}$  itself to a search problem.

## 6.3 Meta-Binary Search

**Definition 6.1** (Meta-Binary Search). A *meta-binary search* is a recursive search architecture in which the directional predicate required by the current binary-search level is itself computed by another instance of an oracle-free search process.

Schematically,

$$\mathcal{B}_0 \text{ needs } \mathcal{D}_0.$$

Instead of supplying  $\mathcal{D}_0$  externally, define

$$\mathcal{D}_0 := \mathcal{B}_1,$$

where  $\mathcal{B}_1$  is itself an oracle-free binary-search instance constructed to determine the answer required by  $\mathcal{B}_0$ .

But now  $\mathcal{B}_1$  requires its own directional predicate

$$\mathcal{D}_1.$$

Thus

$$\mathcal{D}_1 := \mathcal{B}_2,$$

and so on.

The resulting tower is

$$\boxed{\mathcal{B}_0 \leftarrow \mathcal{B}_1 \leftarrow \mathcal{B}_2 \leftarrow \cdots}$$

where each level attempts to supply the missing direction of the previous level.

## 6.4 Self-Oracularization

The preceding recursion motivates a more general concept.

**Definition 6.2** (Self-Oracularization). Let  $A$  be an algorithmic process requiring access to an oracle  $O$ . A *self-oracularization* of  $A$  is a construction in which each query to  $O$  is replaced by an encoded instance of the same computational framework, recursively tasked with determining the oracle answer.

In symbolic form,

$$A^O(x)$$

is replaced by

$$A^{A^{A^{\cdots}}}(x).$$

The purpose is not to execute an actually infinite recursion. The purpose is to identify a finite representation of its self-similar dependency structure.

This leads to the next object.

## 6.5 The Infinite Oracle Tower

Let

$$Q_0$$

be the original query.

Suppose answering  $Q_0$  requires a subsidiary query

$$Q_1.$$

Answering  $Q_1$  requires

$$Q_2,$$

and so forth.

The naive dependency structure is

$$Q_0 \rightarrow Q_1 \rightarrow Q_2 \rightarrow Q_3 \rightarrow \cdots .$$

If interpreted operationally, this never reaches a base case.

The intended construction instead treats the infinite tower as a *self-similar object*.

If every level is generated by the same transformation

$$Q_{i+1} = T(Q_i),$$

then the entire tower is determined by

$$Q_0 \quad \text{and} \quad T.$$

Thus the information content of the infinite sequence need not equal the length of the sequence.

This distinction is crucial.

An infinite object may admit a finite description whenever its structure is recursive, periodic, algebraic, automorphic, or otherwise generated by a finite rule.

## 6.6 Holographic Encoding

We use the term *holographic encoding* for the intended finite representation of this recursive dependency structure.

**Definition 6.3** (Holographic Oracle Encoding). A *holographic oracle encoding* is a finite representation

$$H(Q, T)$$

of an unbounded recursively generated oracle-dependency tower

$$Q, T(Q), T^2(Q), \dots$$

such that the global consistency conditions of the tower can be recovered from polynomially many explicitly represented relations.

The term *holographic* is used here in the structural sense that the finite representation is intended to encode global information through distributed local constraints.

This is conceptually reminiscent of redundant proof representations such as those appearing in probabilistically checkable proofs: global correctness may be constrained by many locally testable relations [6]. The present construction, however, is not identified with the PCP theorem itself. Rather, PCP serves as evidence that highly redundant encodings can expose global consistency through structured local access.

The desired encoding has the schematic form

|                                                                                   |
|-----------------------------------------------------------------------------------|
| infinite recursive oracle tower $\longrightarrow$ finite self-consistency object. |
|-----------------------------------------------------------------------------------|

## 6.7 Fixed-Point Formulation

A natural way to collapse the infinite recursion is through a fixed point.

Suppose the oracle state is represented by some object  $h$ , and one level of self-oracularization induces a transformation

$$\mathcal{F}_x(h).$$

A self-consistent oracle representation should satisfy

$$h^* = \mathcal{F}_x(h^*).$$

Thus the infinite dependency tower is replaced by the finite equation

$$\boxed{h = \mathcal{F}_x(h).}$$

The role of the holographic representation is to encode this equation and its consistency constraints in polynomial space.

The computational objective then becomes:

$$\boxed{\text{solve the self-consistency condition rather than unfold the recursion.}}$$

This is the key meta-search move.

## 6.8 Polynomial Compression Requirement

For the construction to be useful, the fixed-point representation must not inherit exponential size.

Let  $n$  denote the size of the original problem instance. We require an encoding

$$H_x$$

such that

$$|H_x| \leq n^{O(1)}.$$

The strongest intended form is near-linear or subquadratic representation,

$$|H_x| = O(n^c)$$

for a small constant  $c$ , ideally close to 1.

The encoding must therefore exploit repeated structure rather than explicitly store each recursion level.

If the same relation recurs at every depth, then one stores the generating rule once and imposes a global fixed-point condition.

## 6.9 Toy Example: Meta-Binary Search

Consider a binary search that needs to determine whether the target belongs to the left region  $L$  or right region  $R$ .

Ordinarily one computes

$$D(I) = \begin{cases} 0, & z \in L, \\ 1, & z \in R. \end{cases}$$

Suppose  $D$  is unavailable.

Meta-binary search replaces the query “what is  $D(I)$ ?” by a second search whose target is the answer to that very directional question.

Then the second search requires its own directional answer, producing

$$D_0 \leftarrow D_1 \leftarrow D_2 \leftarrow \cdots$$

If every  $D_i$  is generated by the same local rule, the entire sequence may be described by a finite transition relation

$$D_{i+1} = T(D_i).$$

Rather than expanding the sequence, introduce a global encoded state  $h$  satisfying

$$h = T(h).$$

The toy problem is already in  $\mathbf{P}$ , so this construction provides no new complexity-theoretic consequence. Its purpose is only to isolate the mechanism:

replace a missing oracle by recursive self-reference, then compress the self-reference globally.

The relevant question is whether the same architecture can be transferred to an NP-complete problem without transferring the computational difficulty into the encoding.

## 6.10 From Meta-Binary Search to SAT

We now apply the same idea to Boolean satisfiability.

Let

$$F$$

be a CNF formula.

A guided SAT procedure would like access to a directional predicate such as

$$O_F(u) = \begin{cases} 0, & \text{a satisfying completion remains below } u0, \\ 1, & \text{otherwise choose } u1. \end{cases}$$

As emphasized earlier, directly computing this oracle is equivalent to solving a restricted SAT question and therefore cannot simply be assumed.

Instead, construct the oracle query itself as another SAT instance.

That is, let

$$F_0 := F.$$

The missing directional query for  $F_0$  is encoded as a new formula

$$F_1 := \mathcal{O}(F_0),$$

where  $F_1$  represents the statement needed to determine the direction of the search on  $F_0$ .

But  $F_1$  is itself a SAT instance and therefore requires its own missing directional information.

Construct

$$F_2 := \mathcal{O}(F_1),$$

then

$$F_3 := \mathcal{O}(F_2),$$

and recursively

$$F_{i+1} = \mathcal{O}(F_i).$$

The resulting structure is

$$\boxed{F_0 \leftarrow F_1 \leftarrow F_2 \leftarrow F_3 \leftarrow \dots}$$

where each SAT instance is intended to encode the oracle information required by the preceding SAT instance.

## 6.11 SAT Self-Oracularization

**Definition 6.4** (SAT Self-Oracularization). A *SAT self-oracularization* is a recursively defined family

$$F_{i+1} = \mathcal{O}(F_i)$$

such that satisfying information for  $F_{i+1}$  determines a directional decision required to navigate the witness space of  $F_i$ .

The infinite tower is not intended to be explicitly materialized.

Instead, the goal is to represent the entire recursion by a finite self-consistency object

$$H_F.$$

The desired equivalence is

$$\boxed{H_F \equiv \{F_0, F_1, F_2, \dots\}}$$

in the sense that solving the consistency constraints encoded in  $H_F$  reveals the directional information that the infinite tower would provide.

## 6.12 Holographic SAT State

We therefore introduce a global state

$$H_F$$

containing a polynomial-size representation of:

1. the original formula  $F$ ;
2. the rule  $\mathcal{O}$  generating oracle-SAT instances;
3. the consistency relation connecting one oracle level to the next;
4. a finite representation of the self-similar recursion;
5. a decoding map producing a directional decision for  $F$ .

Formally, the target is a polynomial-time constructible object

$$H_F = \text{Holo}(F, \mathcal{O})$$

with

$$|H_F| \leq |F|^{O(1)}.$$

The representation should satisfy a fixed-point relation of the form

$$H_F = \mathcal{G}(F, H_F).$$

Thus SAT is not supplied with an external oracle. It is supplied with an encoded representation of its own recursively defined oracle obligations.

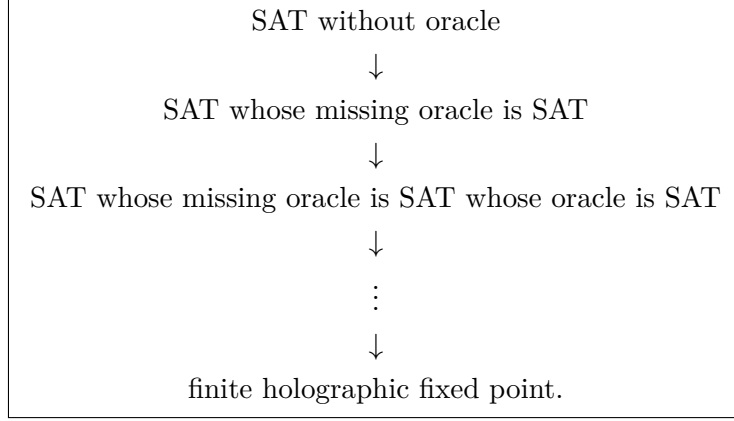
This is the NP analogue of meta-binary search.

## 6.13 The Meta-SAT Principle

The central construction can now be stated succinctly.

**Principle 6.5** (Meta-SAT Principle). *When a SAT procedure requires a directional oracle, encode the oracle query as another SAT instance of the same general type. Repeat this construction recursively, and replace the resulting infinite self-reference by a finite polynomial-size fixed-point representation.*

Schematically,



The intended effect is to transform missing direction into a global self-consistency condition.

### 6.14 Connection to the $\Omega$ Property

This construction suggests a concrete interpretation of the global property  $\Omega$  introduced in Section 5.

Rather than viewing  $\Omega$  as a scalar score, define

$$\Omega(F)$$

to be the globally self-consistent state induced by the recursively self-oracularized SAT representation.

In this interpretation,

$$\Omega(F) = \text{Fix}(\mathcal{G}_F),$$

where

$$\mathcal{G}_F$$

is the finite operator encoding one level of recursive oracle expansion.

The direction function is then extracted by a decoder

$$D_F = \text{Dec}(\Omega(F)).$$

Thus the proposed chain becomes

|                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------|
| $F \longrightarrow H_F \longrightarrow \Omega(F) \longrightarrow D_F \longrightarrow \text{guided SAT search.}$ |
|-----------------------------------------------------------------------------------------------------------------|

This provides the first explicit architectural candidate for  $\Omega$ .

### 6.15 Why Infinite Recursion Does Not Automatically Imply Infinite Size

The entire construction depends on one distinction:

|                                            |
|--------------------------------------------|
| recursive depth $\neq$ description length. |
|--------------------------------------------|

For example, the infinite sequence

$$1, 1, 1, 1, \dots$$

has infinite length but constant descriptive complexity.

Likewise,

$$x_{i+1} = f(x_i)$$

may describe infinitely many states using only the finite pair

$$(x_0, f).$$

The self-oracularization program seeks an analogous compression for recursive SAT oracle dependencies.

The challenge is therefore not to “store infinity.” The challenge is to find a finite relation whose fixed points exactly encode the consistency of the infinite dependency structure.

## 6.16 The Compression Lemma Target

The next mathematical target is the following.

**Conjecture 6.6** (Holographic Compression Lemma). *For every SAT instance  $F$ , the recursively generated self-oracle tower*

$$F, \mathcal{O}(F), \mathcal{O}^2(F), \dots$$

*admits a representation*

$$H_F$$

*of size*

$$|H_F| \leq |F|^c$$

*for some uniform constant  $c$ , such that global consistency of  $H_F$  determines the directional oracle answer required for  $F$ .*

A stronger desired form would be

$$|H_F| = O(|F|)$$

or

$$|H_F| = O(|F| \log |F|).$$

The exact exponent is secondary at this stage. The essential requirement is a uniform polynomial bound.

## 6.17 The Computation Lemma Target

Compression alone is insufficient. The encoded fixed point must also be computable.

**Conjecture 6.7** (Polynomial Fixed-Point Extraction). *There exists a deterministic polynomial-time procedure*

$$\mathcal{H}$$

*such that*

$$\mathcal{H}(H_F)$$

*computes a globally consistent state*

$$\Omega(F)$$

*from which at least one correct directional SAT decision can be decoded.*

If both the compression and extraction conjectures are established, then the missing SAT oracle is no longer assumed externally.

Instead,

SAT constructs a finite representation of its own oracle dependency,  
then solves the resulting global consistency object.

## 6.18 No-Oracle Closure

The intended construction may therefore be represented as

oracle demand  $\rightarrow$  self-reference  $\rightarrow$  finite fixed point  $\rightarrow$  direction.

This is the key distinction from ordinary oracle computation.

No external decision oracle is introduced.

The oracle is replaced by a self-consistency condition internal to the NP-native construction.

We call this property *no-oracle closure*.

**Definition 6.8** (No-Oracle Closure). An NP-native search construction has no-oracle closure if every directional query generated during the computation is recursively represented inside the same finite global state, and the final directional information is obtained by solving the consistency conditions of that state rather than by invoking an external decision oracle.

## 6.19 Section Target

The architectural target is therefore

Meta-binary search:

missing direction  $\rightarrow$  recursive self-direction  $\rightarrow$  finite fixed point,

Meta-SAT:

missing SAT oracle  $\rightarrow$  recursive SAT self-oracle  $\rightarrow$  holographic polynomial state.

The next section must make the operator

$$\mathcal{O}$$

explicit, define the finite encoding

$$H_F,$$

and prove that its size and fixed-point extraction cost are polynomial.

## 7 Iterated Function Systems as a Finite Model of Infinite Self-Oracularization

### 7.1 Why Iterated Function Systems Appear Naturally

The previous section produced an infinite recursively self-oracular structure,

$$F_0, F_1, F_2, \dots, \quad F_{i+1} = \mathcal{O}(F_i),$$

and proposed that the entire tower should be replaced by a finite self-consistency object.

Iterated Function Systems provide a natural mathematical language for exactly this type of phenomenon: a finite collection of transformations can generate an infinite self-similar object whose global form is characterized as a fixed point [5].

Let  $(X, d)$  be a complete metric space and let

$$\mathfrak{F} = \{f_1, f_2, \dots, f_r\}$$

be a finite family of contractions

$$f_i : X \rightarrow X.$$

The associated Hutchinson operator acts on subsets  $A \subseteq X$  by

$$\mathcal{H}_{\mathfrak{F}}(A) := \bigcup_{i=1}^r f_i(A).$$

Under the standard contractive hypotheses, there exists a unique compact attractor  $K$  satisfying

$$K = \mathcal{H}_{\mathfrak{F}}(K) = \bigcup_{i=1}^r f_i(K).$$

Thus a finite collection of maps specifies an object containing the limit of arbitrarily deep compositions

$$f_{i_1} \circ f_{i_2} \circ \dots \circ f_{i_k}, \quad k \rightarrow \infty.$$

This is precisely the representational phenomenon required by the self-oracular construction.

### 7.2 Finite Rule, Infinite Structure

An IFS separates two quantities that must not be confused:

depth of generated structure      and      size of its description.

Although the attractor contains information arising from arbitrarily long compositions, its generating description contains only the finite family

$$f_1, \dots, f_r.$$

Hence

infinite recursive depth  $\not\Rightarrow$  infinite description length.

This gives a concrete mathematical realization of the “holographic” principle introduced in Section 6.

### 7.3 A Binary IFS on Cantor Space

Boolean satisfiability is naturally associated with binary strings. It is therefore useful to work not initially in Euclidean space, but in binary Cantor space

$$\Sigma^\omega := \{0, 1\}^{\mathbb{N}}.$$

For distinct sequences  $x, y \in \Sigma^\omega$ , let

$$\ell(x, y)$$

be the index of their first differing bit, and define the ultrametric

$$d(x, y) := 2^{-\ell(x, y)}.$$

Now define the prefix maps

$$P_0(x) = 0x, \quad P_1(x) = 1x.$$

These satisfy

$$d(P_b(x), P_b(y)) = \frac{1}{2}d(x, y), \quad b \in \{0, 1\}.$$

Thus each prefix operation is a contraction with factor  $1/2$ .

This observation gives an immediate bridge between

binary witness trees

and

contractive iterated function systems

A finite Boolean assignment

$$a_1 a_2 \cdots a_k$$

corresponds to the composition

$$P_{a_1} \circ P_{a_2} \circ \cdots \circ P_{a_k}.$$

Therefore an exponentially large binary search tree can be described through a constant-size family of generating maps.

## 7.4 From the Assignment Tree to a SAT-Conditioned IFS

The unrestricted binary IFS

$$\{P_0, P_1\}$$

generates the entire binary space and therefore contains no SAT-specific directional information.

The required construction must condition the maps on the formula.

Let  $F$  be a CNF formula. We introduce a finite formula-dependent family

$$\mathfrak{F}_F = \{f_1^{(F)}, f_2^{(F)}, \dots, f_r^{(F)}\}$$

acting on a polynomially describable metric state space

$$\mathcal{X}_F.$$

A point

$$x \in \mathcal{X}_F$$

may encode, for example,

$$x = (u, \sigma, \eta),$$

where

$$u$$

is a partial assignment,

$$\sigma$$

is a compact formula state, and

$$\eta$$

is the recursively generated oracle-consistency state.

Each map

$$f_i^{(F)}$$

represents one admissible local transformation of this NP-native state.

The corresponding global operator is

$$\mathcal{H}_F(A) := \bigcup_{i=1}^r f_i^{(F)}(A).$$

The proposed global state is the fixed attractor

$$\boxed{K_F = \mathcal{H}_F(K_F).}$$

## 7.5 Identification of $\Omega$

We now obtain a concrete candidate for the abstract  $\Omega$ -property introduced earlier.

**Definition 7.1** (IFS  $\Omega$ -State). For a SAT instance  $F$ , define

$$\Omega(F) := K_F,$$

where  $K_F$  is the invariant set of the formula-conditioned IFS  $\mathfrak{F}_F$ .

The intended meaning is that  $K_F$  simultaneously represents all recursively consistent continuations generated by the finite self-oracular rule.

Instead of explicitly storing

$$F_0, F_1, F_2, \dots,$$

the construction stores a finite description of

$$\mathfrak{F}_F,$$

while the fixed-point equation

$$K_F = \bigcup_i f_i^{(F)}(K_F)$$

represents the closure of the complete recursion.

Thus the earlier schematic identity

$$\Omega(F) = \text{Fix}(\mathcal{G}_F)$$

is realized by the IFS equation

$$\boxed{\Omega(F) = \text{Fix}(\mathcal{H}_F).}$$

## 7.6 Recursive Oracle Queries as IFS Addresses

Every infinite sequence of map indices

$$\alpha = i_1 i_2 i_3 \dots$$

is an *address* in the symbolic space of the IFS.

Starting from any state  $x_0$ , the sequence

$$\begin{aligned} & f_{i_1}^{(F)}(x_0), \\ & f_{i_1}^{(F)} \circ f_{i_2}^{(F)}(x_0), \\ & f_{i_1}^{(F)} \circ f_{i_2}^{(F)} \circ f_{i_3}^{(F)}(x_0), \end{aligned}$$

and so forth, encodes arbitrarily deep recursive choices.

This allows the oracle tower

$$F_0 \leftarrow F_1 \leftarrow F_2 \leftarrow \dots$$

to be reinterpreted as an address-selection process in a single invariant global object.

The recursion depth is moved from explicit storage into composition depth.

The entire family of depths is represented at once by the attractor.

## 7.7 The Oracle Attractor

We therefore introduce the central object of this section.

**Definition 7.2** (Oracle Attractor). The *oracle attractor* of a SAT instance  $F$  is a fixed invariant set

$$K_F$$

of a finite formula-conditioned IFS such that consistent addresses of  $K_F$  encode recursively valid oracle answers for the SAT self-oracularization process.

The desired decoder has the form

$$\text{Dir}_F(K_F, u) \in \{0, 1, \perp\},$$

where

$$0$$

means that the  $u0$  branch should be preserved,

$$1$$

means that the  $u1$  branch should be preserved, and

$$\perp$$

denotes a terminal or inconsistent state.

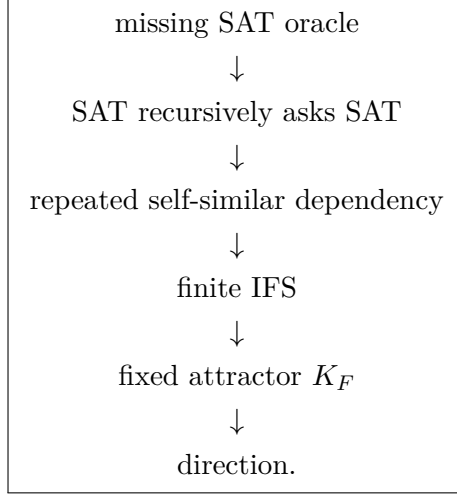
If

$$\text{Dir}_F(K_F, u)$$

can be computed in polynomial time, the attractor acts as the internally generated directional oracle required by the previous sections.

## 7.8 Self-Oracularization as a Fractal Fixed Point

The conceptual chain now becomes



The infinite self-oracularization is therefore not treated as an infinite algorithm. It is treated as the invariant object of a finite dynamical system.

## 7.9 The Holographic Interpretation

The word *holographic* can now be made less metaphorical.

A small set of generating transformations

$$f_1^{(F)}, \dots, f_r^{(F)}$$

determines an arbitrarily detailed global structure.

Every finite-depth view corresponds to a finite composition.

The entire infinite closure is characterized by one fixed-point equation.

Thus information is represented in three layers:

|                    |                             |
|--------------------|-----------------------------|
| generator level:   | $\mathfrak{F}_F,$           |
| global level:      | $K_F = \mathcal{H}_F(K_F),$ |
| local query level: | $\text{Dir}_F(K_F, u).$     |

This is the proposed holographic architecture.

## 7.10 Description Complexity

The attractor itself need not be explicitly enumerated.

The representation size is defined by the description of the generating system:

$$\text{size}(K_F) := \text{size}(\mathfrak{F}_F).$$

If the number of maps satisfies

$$r \leq |F|^{c_1}$$

and each map has a description of length at most

$$|F|^{c_2},$$

then

$$\text{size}(\mathfrak{F}_F) \leq |F|^{c_1+c_2}.$$

Thus the entire infinitely generated object has a polynomial-size description.

A stronger target is

$$r = O(1)$$

with map descriptions of size

$$O(|F| \log |F|),$$

which would yield a near-linear holographic representation.

### 7.11 Why Compression Alone Is Not Enough

The IFS immediately supplies a language for finite description of infinite self-similarity. The decisive problem is now shifted into the structure of the maps.

The maps must satisfy four properties simultaneously:

- (i) finite/poly-size description,
- (ii) contractive or otherwise uniquely convergent dynamics,
- (iii) SAT-faithful fixed-point semantics,
- (iv) polynomial-time extraction of direction.

These are no longer vague requirements. They define the exact mathematical lemmas needed by the construction.

### 7.12 IFS Compression Lemma Target

**Conjecture 7.3** (SAT-IFS Compression Lemma). *There exists a polynomial-time transformation that maps every CNF formula  $F$  to a finite IFS*

$$\mathfrak{F}_F$$

*whose description length satisfies*

$$|\mathfrak{F}_F| \leq |F|^{O(1)},$$

*and whose invariant state  $K_F$  encodes the complete recursively self-oracular SAT dependency structure.*

This replaces the earlier abstract Holographic Compression Lemma by a specific dynamical-system formulation.

### 7.13 Contractivity Lemma Target

**Conjecture 7.4** (Uniform Contractivity). *The maps of  $\mathfrak{F}_F$  can be chosen so that there exists a uniform constant*

$$0 < \lambda < 1$$

with

$$d(f_i^{(F)}(x), f_i^{(F)}(y)) \leq \lambda d(x, y)$$

for every formula  $F$ , every map  $f_i^{(F)}$ , and all valid states  $x, y \in \mathcal{X}_F$ .

A uniform contraction factor is particularly useful because approximation to the invariant state then converges geometrically.

If the initial error is  $E_0$ , after  $k$  iterations the error is bounded schematically by

$$E_k \leq \lambda^k E_0.$$

Thus accuracy

$$E_k \leq \varepsilon$$

requires only

$$k = O\left(\log \frac{1}{\varepsilon}\right)$$

iterations when  $\lambda$  is uniformly bounded away from 1.

### 7.14 Directional Decoding Lemma Target

The final bridge back to SAT is a decoder.

**Conjecture 7.5** (Polynomial Directional Decoding). *There exists a deterministic polynomial-time procedure*

Dir

*such that, for every satisfiable formula  $F$  and every extendible partial assignment  $u$ ,*

$$\text{Dir}(F, K_F, u)$$

*returns a bit  $b \in \{0, 1\}$  for which*

$ub$

*still has at least one satisfying completion.*

If this conjecture holds, one may construct a full satisfying assignment in at most  $n$  directional steps.

## 7.15 The Complete IFS Route

The proposed algorithmic route is now

$$F \mapsto \mathfrak{F}_F \mapsto K_F \mapsto b_1 \mapsto b_2 \mapsto \cdots \mapsto b_n.$$

The resulting witness is

$$w^\star = b_1 b_2 \cdots b_n.$$

If

$$|\mathfrak{F}_F| \leq |F|^{O(1)},$$

the attractor information needed by the decoder is computable to sufficient precision in polynomial time, and each directional bit is correct, then the complete procedure runs in polynomial time.

The IFS therefore supplies a candidate realization of all three missing objects from the previous sections:

$$\begin{aligned} H_F &\equiv \mathfrak{F}_F, \\ \Omega(F) &\equiv K_F, \\ D_F &\equiv \text{Dir}(F, K_F, \cdot). \end{aligned}$$

## 7.16 The Seventh Principle

**Principle 7.6** (IFS Holographic Closure). *Represent recursive SAT self-oracularization not by explicitly unfolding the oracle tower, but as the invariant state of a finite formula-conditioned iterated function system.*

In its shortest form,

$$\text{infinite SAT recursion} \longrightarrow \text{finite IFS} \longrightarrow \text{global attractor} \longrightarrow \text{direction.}$$

The next stage must therefore define the actual maps

$$f_i^{(F)}$$

and the state metric

$$d_F$$

for Boolean satisfiability.

# 8 Fourier Holography and Global Spectral Closure

## 8.1 Why a Second Holographic Layer Is Needed

The iterated-function-system construction of Section 7 provides a finite generative description of an infinitely recursive self-oracular structure. However, a finite description alone does not yet provide an efficient method for extracting the global direction required by SAT.

The next step is therefore to represent the fixed self-similar state in a basis in which global structure is explicitly distributed across a finite collection of spectral coefficients.

This motivates a second layer of the construction:

$$\boxed{\text{IFS holography} \quad + \quad \text{Fourier holography.}}$$

The IFS layer compresses recursive depth.

The Fourier layer reorganizes global dependence.

## 8.2 Fourier Representation of Boolean State Space

Let

$$f : \{-1, +1\}^n \rightarrow \mathbb{R}$$

be any real-valued function defined on Boolean assignments.

For every subset

$$S \subseteq [n],$$

define the parity character

$$\chi_S(x) := \prod_{i \in S} x_i.$$

The Fourier–Walsh expansion of  $f$  is [7]

$$f(x) = \sum_{S \subseteq [n]} \hat{f}(S) \chi_S(x),$$

with coefficients

$$\hat{f}(S) = 2^{-n} \sum_{x \in \{-1, +1\}^n} f(x) \chi_S(x).$$

Thus a global Boolean landscape is represented by its spectral signature

$$\hat{f} = \{\hat{f}(S)\}_{S \subseteq [n]}.$$

The low-order coefficients describe local or low-degree correlations.

Higher-order coefficients encode larger-scale interactions among variables.

The complete spectrum therefore contains a global description of the assignment landscape.

## 8.3 The SAT Landscape as a Signal

Let  $F$  be a CNF formula and define the Boolean satisfaction signal

$$s_F(x) := \begin{cases} 1, & x \models F, \\ 0, & x \not\models F. \end{cases}$$

Equivalently, one may use the signed signal

$$g_F(x) := 2s_F(x) - 1.$$

The SAT witness space is then represented spectrally by

$$\widehat{g}_F(S).$$

The key observation is not that the full Fourier spectrum can be naively computed efficiently—it contains  $2^n$  coefficients—but that the spectral domain gives a precise language for asking whether the information required for direction can be compressed into a polynomially representable subset, recurrence, transform, or fixed-point relation.

## 8.4 Fourier Holographic State

We introduce the following object.

**Definition 8.1** (Fourier Holographic State). Let  $K_F$  denote the IFS oracle attractor associated with the SAT instance  $F$ . A *Fourier holographic state* is a finite spectral representation

$$\widehat{\Omega}_F$$

whose coefficients encode the global self-consistency relations of  $K_F$  and from which the directional information required by the SAT search can be decoded.

The term *holographic* refers to the intended distribution of global information across the spectral representation: no single local coefficient need encode the entire solution, while the collectively consistent spectrum represents the global state.

The proposed chain is now

$$F \longrightarrow \mathfrak{F}_F \longrightarrow K_F \longrightarrow \widehat{\Omega}_F \longrightarrow D_F.$$

## 8.5 Spectralization of the IFS

Suppose the formula-conditioned IFS contains maps

$$f_1^{(F)}, \dots, f_r^{(F)}.$$

The invariant state satisfies

$$K_F = \bigcup_{i=1}^r f_i^{(F)}(K_F).$$

Rather than reasoning directly about the recursively generated geometry, associate to  $K_F$  a signal or measure

$$\mu_F.$$

For example,  $\mu_F$  may assign weight to recursively consistent states of the self-oracular construction.

The IFS induces an operator

$$\mathcal{T}_F \mu := \sum_{i=1}^r p_i (f_i^{(F)})_{\#} \mu,$$

where

$$p_i \geq 0, \quad \sum_i p_i = 1,$$

and  $(f_i^{(F)})_{\#} \mu$  denotes the pushforward measure.

A stationary holographic state satisfies

$$\mu_F = \mathcal{T}_F \mu_F.$$

Applying a Fourier transform gives a spectral fixed-point equation

$$\boxed{\widehat{\mu}_F = \widehat{\mathcal{T}_F \mu_F}.$$

When the maps possess suitable algebraic form, the transformed equation may reduce geometric self-similarity to algebraic recurrences among Fourier coefficients.

This is the intended role of Fourier holography.

## 8.6 Affine IFS Example

Suppose, for illustration, that the state space is embedded in  $\mathbb{R}^d$  and each IFS map has affine form

$$f_i(x) = A_i x + b_i.$$

If  $\mu$  is a probability measure and

$$\widehat{\mu}(\xi) := \int e^{-2\pi i \langle \xi, x \rangle} d\mu(x),$$

then the pushforward relation gives

$$\widehat{(f_i)_{\#} \mu}(\xi) = e^{-2\pi i \langle \xi, b_i \rangle} \widehat{\mu}(A_i^T \xi).$$

Hence an invariant measure satisfying

$$\mu = \sum_i p_i (f_i)_{\#} \mu$$

obeys

$$\boxed{\widehat{\mu}(\xi) = \sum_i p_i e^{-2\pi i \langle \xi, b_i \rangle} \widehat{\mu}(A_i^T \xi).}$$

The infinite geometric recursion has therefore become a functional equation in frequency space.

This is the first explicit mathematical mechanism by which the IFS recursion may be globally closed without unfolding every depth.

## 8.7 Boolean Fourier Version

For SAT it is often more natural to remain entirely on the Boolean cube.

Let

$$h_F : \{-1, +1\}^m \rightarrow \mathbb{R}$$

encode the global state of the self-oracular construction.

Its Walsh–Hadamard spectrum is

$$\hat{h}_F(S) = 2^{-m} \sum_x h_F(x) \chi_S(x).$$

An NP-native update

$$h \mapsto \mathcal{G}_F(h)$$

induces a spectral update

$$\hat{h} \mapsto \widehat{\mathcal{G}_F(h)}.$$

A holographic fixed point then satisfies

$$\boxed{\hat{h}^* = \widehat{\mathcal{G}_F(h^*)}.$$

The objective is to choose the state representation and update rule so that this transformed fixed-point equation closes over only polynomially many spectral statistics.

## 8.8 Spectral Closure

This motivates the central definition.

**Definition 8.2** (Polynomial Spectral Closure). A SAT holographic state admits *polynomial spectral closure* if there exists a family of spectral observables

$$\Gamma_F = \{\gamma_1, \dots, \gamma_M\}$$

with

$$M \leq |F|^{O(1)}$$

such that:

1. the values of all  $\gamma_i$  are computable from the current holographic state in polynomial time;
2. one application of the self-oracular update transforms  $\Gamma_F$  into a new collection determined solely by polynomial-time functions of the current  $\Gamma_F$ ;

3. the directional decision required by SAT can be decoded from the fixed point of these observables.

Thus we do not require the entire  $2^n$ -coefficient Fourier spectrum.

We require a polynomially closed spectral subsystem.

This is the spectral analogue of the finite IFS description.

## 8.9 The Holographic Closure Equation

Combining the IFS and Fourier layers gives the central fixed-point system

$$\boxed{\begin{aligned} K_F &= \mathcal{H}_F(K_F), \\ \mu_F &= \mathcal{T}_F \mu_F, \\ \widehat{\mu}_F &= \widehat{\mathcal{T}_F \mu_F}. \end{aligned}}$$

The first line describes geometric self-similarity.

The second describes invariant mass or consistency.

The third describes global spectral closure.

The proposed  $\Omega$ -object may therefore be refined as

$$\boxed{\Omega(F) = (\mathfrak{F}_F, K_F, \Gamma_F),}$$

where  $\Gamma_F$  is the polynomially retained Fourier hologram.

## 8.10 Global Information from Distributed Coefficients

The purpose of the Fourier layer is not to assign a scalar energy to each assignment.

Rather, it is to allow global dependence to be represented collectively.

Suppose

$$\Gamma_F = (\gamma_1, \dots, \gamma_M).$$

A directional decoder may take the form

$$D_F(u) = \text{sign}(P_F(u, \gamma_1, \dots, \gamma_M)),$$

where  $P_F$  is a polynomial-time computable expression.

Then the SAT direction does not arise from one local clause score.

It arises from interference among globally distributed spectral components.

This gives a mathematically precise version of the holographic intuition.

### 8.11 Interference as Global Aggregation

Fourier transforms convert superposition in the original domain into coefficient addition in the frequency domain and convert convolution-type structure into multiplication.

This is useful because many local constraints may contribute to the same global spectral mode. Schematically,

$$\text{local clause effects} \longrightarrow \text{spectral components} \longrightarrow \text{global interference pattern}.$$

The desired  $\Omega$ -signal is therefore not necessarily stored at a single location.

It may emerge only after aggregation:

$$\boxed{\Omega = \text{globally consistent interference of polynomially encoded modes.}}$$

### 8.12 Fourier-Holographic Direction

We define the target object.

**Definition 8.3** (Fourier-Holographic Directional Signal). A Fourier-holographic directional signal is a polynomial-time computable function

$$D_{\text{FH}}(F, u, \Gamma_F) \in \{0, 1, \perp\}$$

such that, whenever  $u$  has at least one satisfying completion, the returned bit  $b \in \{0, 1\}$  satisfies

$$\exists v \quad ubv \models F.$$

The central claim required later is therefore not that Fourier analysis automatically solves SAT.

The required theorem is much more specific:

$$\boxed{\text{the self-oracular SAT IFS admits a polynomially closed spectral representation whose fixed point yields a correct directional bit.}}$$

### 8.13 Polynomial Fourier Hologram Target

**Conjecture 8.4** (Polynomial Fourier Hologram). *For every CNF formula  $F$ , there exists a polynomial-time constructible spectral state*

$$\Gamma_F$$

*of size*

$$|\Gamma_F| \leq |F|^{O(1)}$$

*such that  $\Gamma_F$  is closed under the SAT self-oracular IFS update and encodes sufficient global information to recover a solution-preserving direction.*

### 8.14 Spectral Fixed-Point Extraction Target

**Conjecture 8.5** (Polynomial Spectral Fixed-Point Extraction). *The fixed point of the polynomial spectral closure equations associated with  $F$  can be computed to the precision required for correct directional decoding in deterministic time*

$$|F|^{O(1)}.$$

This requirement is essential: a polynomial-size spectral representation is insufficient if obtaining its fixed point requires exponential precision or exponential iteration count.

### 8.15 Precision Separation Requirement

Suppose the decoder distinguishes two directions according to a real-valued spectral observable

$$\Delta_F(u).$$

Correct polynomial-time decoding requires a separation margin

$$|\Delta_F(u)| \geq 2^{-\text{poly}(|F|)}$$

whenever a strict directional decision is required.

Then only polynomially many bits of precision are needed to determine the sign of  $\Delta_F(u)$ .

This produces another explicit proof obligation.

**Conjecture 8.6** (Polynomial Spectral Gap). *Whenever the Fourier-holographic decoder must choose between two distinct solution-preserving branches, its decisive spectral observable has magnitude at least*

$$2^{-\text{poly}(|F|)}.$$

Without such a gap, an exponentially precise representation could hide the complexity inside numerical resolution.

### 8.16 IFS–Fourier Dual Closure

The full architecture now has two mutually reinforcing descriptions.

The geometric description is

$$K_F = \bigcup_i f_i^{(F)}(K_F).$$

The spectral description is

$$\Gamma_F = \mathcal{S}_F(\Gamma_F),$$

where

$$\mathcal{S}_F$$

is the finite spectral update induced by the IFS.

The first explains how infinite recursive depth is generated.

The second is intended to make global consistency algorithmically accessible.

Thus

IFS = finite generator of infinite structure,

while

Fourier holography = finite global encoding of distributed structure.

Together,

recursive self-oracle  $\rightarrow$  IFS attractor  $\rightarrow$  Fourier hologram  $\rightarrow$  direction.

### 8.17 Relationship to the $\Phi$ -Model

The  $\Phi$ -model of Section 5 represents a witness landscape that is locally indistinguishable from blind pseudorandom search.

The Fourier perspective suggests one possible way to violate this idealized blindness.

Even if low-order local statistics resemble randomness, the global spectrum may contain structured correlations.

Thus the desired exception is not necessarily

“SAT has obvious local direction.”

Instead it may be

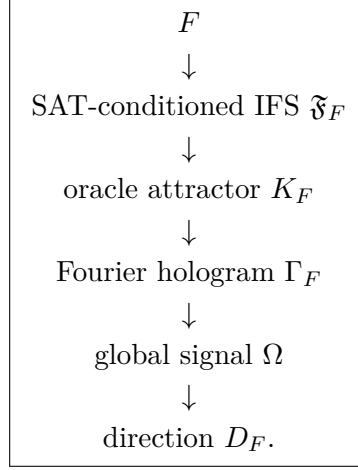
SAT has a globally constrained spectral signature that survives local pseudorandomness.

The role of  $\Omega$  is then precisely to expose this hidden global asymmetry.

### 8.18 The Eighth Principle

**Principle 8.7** (Fourier Holographic Closure). *Encode the invariant state of recursive SAT self-oracularization in a polynomially closed Fourier/Walsh representation, and extract search direction from globally distributed spectral consistency rather than from a purely local score.*

The combined construction is therefore



The remaining task is now sharply defined:

construct the concrete SAT-dependent maps, derive their spectral recurrence,  
and prove polynomial closure and directional correctness.

## 9 The Concrete $\Omega$ Engine: Harmonic Self-Oracles, Weighted IFS, and Fourier Transfer

### 9.1 Choice of Native NP-Complete Language

We now instantiate the abstract architecture of the preceding sections.

The construction will operate on 3-SAT. This choice does not leave the NP-complete setting: 3-SAT is itself NP-complete [4]. Its advantage for the present construction is algebraic. Every clause involves at most three Boolean variables, and therefore every individual clause contributes only constantly many Walsh–Fourier modes.

Let

$$F = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

be a 3-CNF formula over variables

$$x_1, \dots, x_n \in \{-1, +1\}.$$

For every occurrence of  $x_j$  in clause  $C_c$ , let

$$\sigma_{cj} \in \{-1, +1\}$$

encode the sign of the satisfying literal, so that

$$\sigma_{cj}x_j = +1$$

means that this literal is true.

## 9.2 Clause Violation Polynomials

For a clause  $C_c$  containing variables indexed by

$$I_c \subseteq [n], \quad |I_c| \leq 3,$$

define its violation indicator

$$v_c(x) := \prod_{j \in I_c} \frac{1 - \sigma_{cj} x_j}{2}.$$

Then

$$v_c(x) = \begin{cases} 1, & C_c \text{ is violated by } x, \\ 0, & C_c \text{ is satisfied by } x. \end{cases}$$

The global violation energy is

$$E_F(x) := \sum_{c=1}^m v_c(x).$$

Hence

$$x \models F \iff E_F(x) = 0.$$

Because each  $v_c$  involves at most three variables, its Walsh expansion contains at most

$$2^{|I_c|} \leq 8$$

nonzero coefficients.

Thus  $E_F$  itself has a sparse, polynomial-size description in the Walsh basis.

This will be the local algebraic input to the global construction.

## 9.3 The Leaf Satisfaction Signal

Define

$$s_F(x) := \prod_{c=1}^m (1 - v_c(x)).$$

Since each factor belongs to  $\{0, 1\}$ ,

$$s_F(x) = \begin{cases} 1, & x \models F, \\ 0, & x \not\models F. \end{cases}$$

The formula is satisfiable exactly when

$$\sum_{x \in \{-1, +1\}^n} s_F(x) > 0.$$

This signal is the boundary data of the self-oracular system.

## 9.4 The Exact Harmonic Oracle Field

Let

$$u = (u_1, \dots, u_k) \in \{-1, +1\}^k$$

be a partial assignment to the first  $k$  variables in a current variable ordering.

Define the *harmonic oracle amplitude*

$$H_F(u) := 2^{-(n-k)} \sum_{y \in \{-1, +1\}^{n-k}} s_F(uy).$$

Equivalently,

$$H_F(u) = \Pr_Y[F(uY) = 1],$$

where  $Y$  is uniformly distributed over the unassigned variables.

Thus

$$H_F(u) > 0$$

if and only if  $u$  has at least one satisfying completion.

At full depth,

$$H_F(x) = s_F(x).$$

At every internal prefix,

$$H_F(u) = \frac{H_F(u, -1) + H_F(u, +1)}{2}.$$

This identity is exact.

It converts satisfiability direction into a harmonic extension problem on the Boolean prefix tree.

## 9.5 Exact Direction from the Harmonic Field

Define

$$D_F(u) := \begin{cases} -1, & H_F(u, -1) > 0, \\ +1, & H_F(u, -1) = 0 \text{ and } H_F(u, +1) > 0, \\ \perp, & H_F(u, -1) = H_F(u, +1) = 0. \end{cases}$$

Then we immediately obtain the following.

**Lemma 9.1** (Harmonic Direction Lemma). *If*

$$H_F(u) > 0,$$

*then*

$$D_F(u) \in \{-1, +1\}$$

*and the extended prefix*

$$uD_F(u)$$

*has at least one satisfying completion.*

*Proof.* Since

$$H_F(u) = \frac{H_F(u, -1) + H_F(u, +1)}{2} > 0$$

and both child values are nonnegative, at least one child has positive amplitude. The definition of  $D_F$  selects such a child.  $\square$

Consequently, exact access to  $H_F$  gives a satisfying assignment in at most  $n$  directional steps. The problem has therefore been reduced to one sharply defined task:

evaluate the harmonic oracle field without enumerating its leaves.

## 9.6 Meta-Binary Search Reappears

The recurrence

$$H_F(u) = \frac{H_F(u, -1) + H_F(u, +1)}{2}$$

has precisely the form required by the meta-binary-search construction.

To determine the direction at  $u$ , the algorithm asks for the values of two children.

Each child value is defined by two grandchildren.

Each grandchild value is defined by two further descendants.

Hence

$$H_F(u)$$

is the finite-depth form of the recursively self-oracular dependency tower introduced in Section 6.

The naive expansion contains

$$2^{n-k}$$

leaves.

The holographic objective is to represent the complete harmonic field without materializing this tree.

## 9.7 Fourier Identity for the Harmonic Field

Let the Walsh expansion of  $s_F$  be [7]

$$s_F(x) = \sum_{S \subseteq [n]} \widehat{s_F}(S) \chi_S(x),$$

where

$$\chi_S(x) := \prod_{j \in S} x_j.$$

Let

$$P_k := \{1, \dots, k\}$$

be the assigned prefix coordinates.

Taking conditional expectation over the remaining variables annihilates every character containing an unassigned variable. Therefore

$$H_F(u) = \sum_{S \subseteq P_k} \widehat{s_F}(S) \chi_S(u).$$

This identity is exact and provides the bridge from the recursive oracle field to Fourier holography.

The directional query is therefore a spectral query.

## 9.8 Clause Factors in the Walsh Domain

Define the satisfaction factor of clause  $C_c$  by

$$g_c(x) := 1 - v_c(x).$$

Then

$$s_F(x) = \prod_{c=1}^m g_c(x).$$

Let

$$\widehat{g}_c$$

denote the Walsh coefficient vector of  $g_c$ .

Because  $C_c$  has width at most 3,

$$|\text{supp}(\widehat{g}_c)| \leq 8.$$

Multiplication in the Boolean domain becomes XOR-convolution in the Walsh domain. If

$$(a *_{\oplus} b)(S) := \sum_{T \subseteq [n]} a(T) b(S \triangle T),$$

then

$$\widehat{s_F} = \widehat{g}_1 *_{\oplus} \widehat{g}_2 *_{\oplus} \cdots *_{\oplus} \widehat{g}_m.$$

Every individual convolution kernel is constant-sparse.

The exponential object is generated by a polynomial number of constant-local operators.

## 9.9 Local Spectral Transfer Operators

For each clause  $C_c$ , define the linear transfer operator

$$\mathcal{C}_c$$

on spectral functions  $a : 2^{[n]} \rightarrow \mathbb{R}$  by

$$(\mathcal{C}_c a)(S) := \sum_{T \in \text{supp}(\widehat{g}_c)} \widehat{g}_c(T) a(S \triangle T).$$

Since

$$|\text{supp}(\widehat{g}_c)| \leq 8,$$

one output coefficient depends on at most eight input coefficients.

Starting from

$$a_0 = \delta_{\emptyset},$$

define

$$a_c := \mathcal{C}_c a_{c-1}.$$

Then

$$a_m = \widehat{s_F}.$$

Thus

$$\widehat{s_F} = \mathcal{C}_m \mathcal{C}_{m-1} \cdots \mathcal{C}_1 \delta_{\emptyset}.$$

This composition is the first completely explicit form of the Fourier hologram.

## 9.10 The Fourier Hologram Circuit

**Definition 9.2** (Fourier Hologram Circuit). The *Fourier hologram circuit* of  $F$  is the straight-line operator description

$$\Gamma_F := (\mathcal{C}_1, \dots, \mathcal{C}_m)$$

together with the seed

$$\delta_\emptyset.$$

The description length of  $\Gamma_F$  is

$$O(m + n)$$

up to the bit complexity of variable indices and rational coefficients.

Although  $\widehat{s}_F$  may have exponentially many nonzero coefficients, the operator circuit generating it is polynomial in the formula size.

This is the first precise holographic compression:

|                      |                  |                               |
|----------------------|------------------|-------------------------------|
| exponential spectrum | $\longleftarrow$ | polynomial generator circuit. |
|----------------------|------------------|-------------------------------|

No coefficient table is stored.

## 9.11 Weighted IFS Representation of Spectral Expansion

The convolution circuit admits an equivalent path interpretation.

For every clause  $c$  and every

$$T \in \text{supp}(\widehat{g}_c),$$

introduce a transition symbol

$$\tau = (c, T).$$

A path

$$\pi = (T_1, \dots, T_m)$$

determines the terminal frequency

$$S(\pi) := T_1 \triangle T_2 \triangle \dots \triangle T_m$$

and weight

$$W(\pi) := \prod_{c=1}^m \widehat{g}_c(T_c).$$

Then

|                                                   |
|---------------------------------------------------|
| $\widehat{s}_F(S) = \sum_{\pi: S(\pi)=S} W(\pi).$ |
|---------------------------------------------------|

The exponentially many convolution terms have become addresses of a finitely generated weighted symbolic system.

## 9.12 Contractive Address Maps

Choose an integer base

$$B \geq 16.$$

Assign every local transition choice  $T$  an integer code

$$\iota(T) \in \{0, \dots, B-1\}.$$

For every allowed local spectral transition define

$$\phi_T(z) := \frac{z + \iota(T)}{B}, \quad z \in [0, 1].$$

Then

$$|\phi_T(z) - \phi_T(z')| = \frac{1}{B}|z - z'|.$$

Hence every  $\phi_T$  is a contraction with common factor

$$\lambda = \frac{1}{B} < 1.$$

A complete convolution path is encoded by

$$z_\pi = \phi_{T_1} \circ \phi_{T_2} \circ \dots \circ \phi_{T_m}(0).$$

The address coordinate therefore stores an arbitrarily deep local-choice history in a self-similar metric representation.

The algebraic labels

$$S(\pi) \quad \text{and} \quad W(\pi)$$

are deterministic observables of the address.

## 9.13 The Weighted Spectral IFS

**Definition 9.3** (Weighted Spectral IFS). The *weighted spectral IFS* of  $F$  is the collection of contractive address maps  $\phi_T$ , one for each local Walsh mode of every clause, equipped with the label updates

$$S \leftarrow S \Delta T$$

and

$$W \leftarrow W \hat{g}_c(T).$$

The IFS does not explicitly enumerate its paths.

It defines the complete spectral expansion through a finite transition rule.

The Fourier hologram circuit and the weighted IFS are therefore two views of the same object:

|                                                                                     |
|-------------------------------------------------------------------------------------|
| $\Gamma_F \equiv \text{finite operator view} \equiv \text{weighted IFS path view.}$ |
|-------------------------------------------------------------------------------------|

### 9.14 Directional Query Functional

For a prefix  $u$  assigning coordinates  $P_k$ , define the query functional

$$Q_u(a) := \sum_{S \subseteq P_k} a(S) \chi_S(u).$$

By the Fourier identity above,

$$H_F(u) = Q_u(\widehat{s_F}).$$

Hence

$$H_F(u) = Q_u(\mathcal{C}_m \cdots \mathcal{C}_1 \delta_\emptyset).$$

This is the exact algebraic form of the oracle query.

### 9.15 Backward Holographic Evaluation

Instead of propagating the exponentially large spectrum forward, move the query backward through the transfer operators.

Let

$$\mathcal{C}_c^*$$

be the adjoint operator with respect to the standard inner product on spectral functions.

Then

$$\begin{aligned} H_F(u) &= \langle q_u, \mathcal{C}_m \cdots \mathcal{C}_1 \delta_\emptyset \rangle \\ &= \langle \mathcal{C}_1^* \cdots \mathcal{C}_m^* q_u, \delta_\emptyset \rangle, \end{aligned}$$

where

$$q_u(S) = \mathbf{1}_{S \subseteq P_k} \chi_S(u).$$

Therefore only the value at the zero-frequency seed is ultimately required.

The computational question becomes:

Can the backward query state remain polynomially compressed under all adjoint clause transfers?

This is more precise than asking whether the full Fourier spectrum is sparse.

### 9.16 Holographic Quotient States

We now define the compression object.

At transfer depth  $c$ , let

$$q^{(c)}$$

be the backward spectral query state after processing the suffix

$$C_{c+1}, \dots, C_m.$$

Two partial IFS paths  $\pi, \pi'$  are declared equivalent at depth  $c$  if they induce exactly the same action on every future directional query needed by the algorithm.

Write

$$\pi \sim_c \pi'.$$

Let

$$\mathcal{Q}_c$$

be the quotient set of path states under  $\sim_c$ .

A quotient state stores the total interference amplitude of all paths in the same equivalence class.

This is the point at which the IFS and Fourier layers meet:

|                                                                                           |
|-------------------------------------------------------------------------------------------|
| IFS merges by recursive state equivalence;      Fourier merges by interference amplitude. |
|-------------------------------------------------------------------------------------------|

### 9.17 The $\Omega$ State

We can now give a concrete definition of the global object.

**Definition 9.4** (Concrete  $\Omega$ -State). For a formula  $F$  and current prefix  $u$ , define

$$\Omega_F(u) := \left( \Gamma_F, \{\mathcal{Q}_c\}_{c=0}^m, \{A_c(Q)\}_{Q \in \mathcal{Q}_c} \right),$$

where

$$A_c(Q)$$

is the aggregated Fourier interference amplitude of quotient state  $Q$  at depth  $c$ .

The decoder is

$$\text{Dec}(\Omega_F(u)) = H_F(u).$$

The directional operator is then

|                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $D_\Omega(F, u) = \begin{cases} -1, & \text{Dec}(\Omega_F(u, -1)) > 0, \\ +1, & \text{Dec}(\Omega_F(u, -1)) = 0 \text{ and } \text{Dec}(\Omega_F(u, +1)) > 0, \\ \perp, & \text{otherwise.} \end{cases}$ |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

This is a fully specified target interface.

### 9.18 The Central Compression Condition

The only reason the quotient representation can fail to be polynomial is if the number of genuinely distinct spectral future behaviors grows exponentially.

Therefore define

$$q_F := \max_{0 \leq c \leq m} |\mathcal{Q}_c|.$$

The crucial structural condition is

$$q_F \leq |F|^{O(1)}.$$

If this holds uniformly, every backward transfer can be implemented over only polynomially many quotient amplitudes.

This motivates the main engineering lemma.

**Conjecture 9.5** (Holographic Quotient Lemma). *For every 3-CNF formula  $F$ , the weighted spectral IFS admits a direction-preserving quotient representation for which*

$$\max_c |Q_c| \leq |F|^{O(1)},$$

*and the quotient transition relation is constructible in deterministic polynomial time from the clause-incidence description of  $F$ .*

This is the exact place where the proposed construction must outperform explicit decision diagrams, explicit Fourier tables, and naive variable elimination.

### 9.19 No Hidden Precision Cost

All clause Fourier coefficients are dyadic rationals with denominator at most

$$2^3.$$

After processing  $m$  clauses, every exact path amplitude has denominator at most

$$2^{3m}.$$

Hence exact arithmetic requires only

$$O(m)$$

bits in the exponent/denominator representation per multiplicative path.

Aggregated amplitudes may require an additional polynomial number of bits to store their exact integer numerators whenever quotient size is polynomial.

Therefore, conditional on the Holographic Quotient Lemma, the construction does not require infinite real precision.

### 9.20 The Concrete Direction Algorithm

Assuming the quotient state can be maintained in polynomial size, the direction algorithm is now explicit.

A complete solver repeats Algorithm 2 at most  $n$  times.

### 9.21 Conditional Complexity Bound

Let

$$Q(|F|)$$

bound the number of holographic quotient states and let

$$R(|F|)$$

---

**Algorithm 2** Fourier–IFS Holographic SAT Direction

---

**Require:** 3-CNF formula  $F$ , satisfiable prefix  $u$

1: Construct the clause-local Walsh kernels

$$\widehat{g}_1, \dots, \widehat{g}_m$$

2: Construct the polynomial generator circuit

$$\Gamma_F = (\mathcal{C}_1, \dots, \mathcal{C}_m)$$

3: **for**  $b \in \{-1, +1\}$  **do**

4:   Build the directional query state  $q_{ub}$

5:   Propagate  $q_{ub}$  backward through the quotient transfer system

6:   Extract

$$h_b \leftarrow Q_{ub}(\widehat{s_F}) = H_F(ub)$$

7: **end for**

8: **if**  $h_{-1} > 0$  **then**

9:   **return**  $-1$

10: **else if**  $h_{+1} > 0$  **then**

11:   **return**  $+1$

12: **else**

13:   **return**  $\perp$

14: **end if**

---

bound the arithmetic cost per quotient transition.

Then one directional query costs

$$O(m Q(|F|) R(|F|)).$$

At most  $n$  directional queries are required.

Therefore

$$\boxed{T_{\text{SAT}}(F) = O(nm Q(|F|) R(|F|)).}$$

If

$$Q(|F|) \leq |F|^c$$

and

$$R(|F|) \leq |F|^d,$$

then

$$T_{\text{SAT}}(F) \leq |F|^{O(1)}.$$

The entire complexity question has therefore been concentrated into the size and constructibility of the holographic quotient.

## 9.22 Why the Construction Is NP-Native

Every object above is generated directly from the 3-SAT verification structure:

clauses  $\rightarrow$  violation polynomials  $\rightarrow$  Walsh kernels  $\rightarrow$  weighted IFS  $\rightarrow$  harmonic oracle field.

No shortest-path solver, sorting oracle, matching algorithm, or other independent P-complete geometry is used to manufacture direction.

The construction remains inside the Boolean witness space and its native spectral representation.

Thus it satisfies the architectural restriction of Section 4.

## 9.23 Why the Construction Is Holographic

The same global information appears in four equivalent representations:

|                |                              |
|----------------|------------------------------|
| leaf view:     | $s_F(x),$                    |
| harmonic view: | $H_F(u),$                    |
| spectral view: | $\widehat{s}_F,$             |
| IFS view:      | $\{(\pi, S(\pi), W(\pi))\}.$ |

The leaf view contains exponentially many assignments.

The harmonic view contains the complete self-oracle tree.

The spectral view contains exponentially many possible Fourier modes.

The IFS view contains exponentially many paths.

Yet all four arise from the same polynomial-size formula and the same constant-local clause transitions.

The intended holographic quotient identifies repeated global behavior across these representations rather than storing any of them explicitly.

## 9.24 The Engineering Bottleneck

The construction is now concrete enough that its decisive mathematical question can be stated without metaphor:

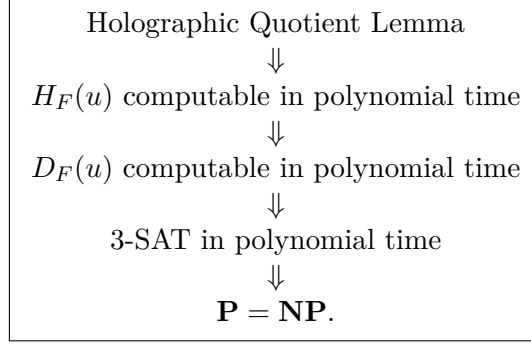
|                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Do the exponentially many weighted Fourier-IFS paths collapse into only polynomially many equivalence classes with respect to all SAT directional queries? |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|

A positive answer supplies polynomial-time access to  $H_F(u)$ .

The Harmonic Direction Lemma then supplies a satisfying branch.

Repeating this for at most  $n$  variables supplies a complete witness.

Thus the chain is



## 9.25 Ninth Principle: Harmonic–Spectral Self-Oracle

**Principle 9.6** (Harmonic–Spectral Self-Oracle). *Use the exact conditional satisfying density*

$$H_F(u)$$

*as the self-oracular directional field; represent its boundary signal by a polynomial clause-local Walsh transfer circuit; encode the exponentially many spectral expansion paths as a weighted contractive IFS; and recover direction through a polynomial-size interference quotient.*

This supplies explicit definitions for

|                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $f_i^{(F)} \equiv \phi_T,$<br>$\mathcal{S}_F \equiv \mathcal{C}_m \cdots \mathcal{C}_1,$<br>$\Gamma_F \equiv (\mathcal{C}_1, \dots, \mathcal{C}_m),$<br>$\Omega_F \equiv \text{quotiented harmonic-spectral IFS state},$<br>$D_F \equiv \text{positivity comparison of child harmonic amplitudes}.$ |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

The abstract architecture has therefore been reduced to a single explicit compression theorem.

## 10 Reflective Closure: Using the Construction to Certify Its Own Compression

### 10.1 The Recursive Proof Idea

The previous section reduced the construction to the Holographic Quotient Lemma: exponentially many Fourier–IFS paths must collapse into only polynomially many future-relevant equivalence classes.

A natural question is whether the same recursive architecture used to solve SAT can also be used to establish the compression property required for its own correctness.

This section introduces precisely that idea.

The intended mechanism is not a circular argument of the form

“the algorithm works because the algorithm says it works.”

Instead, the construction must produce a finite certificate of its own self-consistency, and that certificate must be checkable independently by a polynomial-time verifier.

We call this mechanism *reflective closure*.

## 10.2 Computation and Proof as Parallel State Spaces

Let

$$\mathcal{A}_F$$

denote the holographic SAT algorithm associated with formula  $F$ .

Its computational state is

$$S_F.$$

We introduce a second state space

$$P_F$$

whose elements encode proof obligations about the behavior of  $S_F$ .

Typical obligations include

- the quotient transition is well defined,
- equivalent paths remain equivalent under future updates,
- the number of quotient classes remains polynomial,
- the directional decoder preserves satisfiable extensions,
- the arithmetic representation remains polynomially bounded.

The complete reflective state is therefore

$$R_F := (S_F, P_F).$$

## 10.3 The Reflective Update Operator

Suppose the computational state evolves under

$$S_{t+1} = \Phi_F(S_t).$$

The proof state evolves simultaneously under an operator

$$P_{t+1} = \Pi_F(S_t, P_t).$$

Define the combined update

$$\mathcal{R}_F(S, P) := (\Phi_F(S), \Pi_F(S, P)).$$

A reflectively closed state satisfies

$$\boxed{(S^*, P^*) = \mathcal{R}_F(S^*, P^*)}.$$

Thus the same finite fixed-point architecture that closes the computational self-oracle is also used to close the proof obligations generated by that self-oracle.

## 10.4 Self-Application

The central idea can be written schematically as

$$\boxed{\mathcal{A}(\text{SAT instance})}$$

for ordinary execution, while correctness analysis applies the same structural engine to

$$\boxed{\mathcal{A}(\text{description of } \mathcal{A}).}$$

More precisely, let

$$\langle \mathcal{A} \rangle$$

denote a finite encoding of the algorithm.

The reflective construction forms a verification instance

$$\text{ProofInstance}(F, \langle \mathcal{A} \rangle),$$

whose satisfying objects are certificates that the quotient dynamics of  $\mathcal{A}$  satisfy the required invariants on  $F$ .

The same holographic machinery is then applied to this verification instance.

Hence the architecture becomes

$$\boxed{\mathcal{A} \text{ solves SAT} \quad \text{and} \quad \mathcal{A} \text{ searches for a certificate of the invariants used by } \mathcal{A}.}$$

## 10.5 The Non-Circularity Condition

Self-reference alone does not prove correctness.

To avoid logical circularity, the final certificate must be accepted by a verification relation

$$V_{\text{cert}}(F, \langle \mathcal{A} \rangle, C)$$

whose correctness does not assume the theorem being certified.

The reflective layer is therefore valid only if

$$\boxed{V_{\text{cert}} \text{ is polynomial-time computable and sound independently of } \mathcal{A}.}$$

The role of  $\mathcal{A}$  is to *find* the certificate.

The role of  $V_{\text{cert}}$  is to *check* it.

Thus

$$\text{self-reference} \neq \text{self-justification}.$$

Instead,

self-reference + independent certificate verification = reflective certification.

## 10.6 Certificate for the Quotient Lemma

For a given transfer depth  $c$ , let

$$\mathcal{Q}_c$$

be the proposed quotient states.

A certificate

$$C_c$$

must contain enough information to verify:

1. every generated path state belongs to some quotient class;
2. all members of a quotient class induce identical future behavior for the relevant harmonic directional queries;
3. the quotient transition relation is closed;
4. the number of quotient states satisfies

$$|\mathcal{Q}_c| \leq p(|F|)$$

for a fixed polynomial  $p$ ;

5. aggregated Fourier amplitudes are updated exactly under the quotient transition.

The complete certificate is

$$C_F := (C_0, C_1, \dots, C_m).$$

If each  $C_c$  has polynomial size, then

$$|C_F| \leq |F|^{O(1)}.$$

## 10.7 Recursive Certification of Equivalence

The equivalence relation itself may be defined recursively.

At terminal depth,

$$\pi \sim_m \pi'$$

whenever the two paths contribute identically to the final harmonic query.

At an earlier depth  $c$ , define

$$\pi \sim_c \pi'$$

whenever every admissible local continuation sends  $\pi$  and  $\pi'$  into equivalent states at depth  $c + 1$ , with matching aggregate spectral weights.

Schematically,

$$\pi \sim_c \pi' \iff \forall T, \text{Next}(\pi, T) \sim_{c+1} \text{Next}(\pi', T).$$

Thus the quotient relation is itself recursively defined.

This is exactly the type of object to which the holographic fixed-point architecture can be applied.

## 10.8 Proof IFS

Introduce a finite family of proof-state transformations

$$\mathfrak{P}_F = \{p_1^{(F)}, \dots, p_r^{(F)}\}.$$

Each map encodes one local proof obligation, such as checking one transition, one equivalence preservation rule, or one amplitude identity.

The proof attractor

$$K_F^{\text{proof}}$$

satisfies

$$K_F^{\text{proof}} = \bigcup_i p_i^{(F)} (K_F^{\text{proof}}).$$

Thus the infinite recursive family of proof obligations is represented by a finite self-similar proof system.

The computational and proof attractors coexist:

$$K_F^{\text{comp}} \quad \text{and} \quad K_F^{\text{proof}}.$$

## 10.9 Fourier Representation of the Proof State

The same spectral machinery may be applied to the proof IFS.

Let

$$\Gamma_F^{\text{proof}}$$

be the Fourier/Walsh transfer representation of the recursive proof state.

Then the complete reflective hologram is

$$\Gamma_F^{\text{refl}} = (\Gamma_F^{\text{comp}}, \Gamma_F^{\text{proof}}).$$

Its fixed-point equation is

$$\Gamma_F^{\text{refl}} = \mathcal{S}_F^{\text{refl}}(\Gamma_F^{\text{refl}}).$$

The first component computes directional SAT information.

The second component computes a compressed certificate that the quotient representation used by the first component is closed and sound.

### 10.10 The Self-Certifying Quotient Condition

We can now state the desired reflective property.

**Definition 10.1** (Self-Certifying Holographic Quotient). A holographic quotient is *self-certifying* if the same finite transition system that represents the quotient also generates a polynomial-size certificate  $C_F$  satisfying

$$V_{\text{cert}}(F, \Gamma_F, C_F) = 1.$$

The certificate verifier checks the quotient structure directly.

The solver may discover the certificate recursively, but correctness is accepted only after verification.

### 10.11 Contradiction Strategy Inside the Reflective System

The proof-by-contradiction idea can now be embedded in the reflective state.

Assume that the quotient width is superpolynomial.

Then for some depth  $c$ ,

$$|\mathcal{Q}_c| > p(|F|)$$

for every candidate polynomial bound  $p$ .

This implies the existence of too many mutually distinguishable future behavior signatures.

The proof engine attempts to construct a distinguishing certificate

$$W(\pi, \pi')$$

for each pair of allegedly inequivalent states.

If the Fourier-harmonic observable system has only polynomially many independent future signatures, then sufficiently many path states must collide under all future observables.

The reflective contradiction has the form

|                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>assume superpolynomially many quotient states</p> <p>↓</p> <p>obtain superpolynomially many pairwise distinguishable signatures</p> <p>↓</p> <p>apply the same holographic compression to the signature system</p> <p>↓</p> <p>two allegedly distinct signatures receive the same complete future-observable representation</p> <p>↓</p> <p><math>\pi \sim \pi'</math></p> <p>↓</p> <p><math>\perp</math>.</p> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

In this sense, the algorithm applies its own compression mechanism to the space of possible counterexamples to its compression mechanism.

## 10.12 Recursive Compression of Counterexamples

Let

$$\mathcal{W}_F$$

denote the space of witnesses to failure of the Holographic Quotient Lemma.

A witness may contain two paths

$$\pi, \pi'$$

claimed to be inequivalent despite sharing a candidate compressed state.

The verification question is

$$V_{\text{fail}}(F, \pi, \pi') = 1$$

if some future SAT query distinguishes them.

Instead of enumerating future queries, the reflective construction recursively applies its own global query compression to

$$V_{\text{fail}}.$$

Thus

|                                                                            |
|----------------------------------------------------------------------------|
| compression theorem is tested by compressing its own counterexample space. |
|----------------------------------------------------------------------------|

This is the recursive proof mechanism.

### 10.13 Reflective Fixed Point

Let

$$\mathcal{C}_F$$

denote the computation transformer and

$$\mathcal{P}_F$$

the proof transformer.

Define

$$\mathcal{R}_F := \mathcal{C}_F \times \mathcal{P}_F.$$

The desired reflective state satisfies

$$\boxed{R_F^* = \mathcal{R}_F(R_F^*)}.$$

The fixed point contains simultaneously:

- the compressed computational quotient,
- the directional harmonic data,
- the compressed counterexample space,
- the quotient certificate,
- the proof that the certificate verifier accepts.

Only the last item must bottom out in primitive, independently checkable identities.

Examples include exact rational equalities, finite transition-table consistency, clause-local Walsh identities, and explicit cardinality bounds.

### 10.14 Primitive Proof Base

To prevent infinite justificatory regress, reflective proof expansion must terminate semantically at a finite primitive base.

Define

$$\mathcal{B}_{\text{proof}}$$

to contain only claims whose verification requires no recursive theorem invocation, such as:

- integer equality,
- rational equality,
- set membership in an explicitly given finite set,
- one-step transition correctness,
- one clause's Walsh expansion,
- one quotient-class representative check.

The proof attractor compresses infinitely many recursive proof obligations, but every local leaf type belongs to this finite primitive base.

Hence the logical structure is

recursive proof graph  $\longrightarrow$  finite fixed-point certificate  $\longrightarrow$  primitive checks.

### 10.15 Reflective Soundness Target

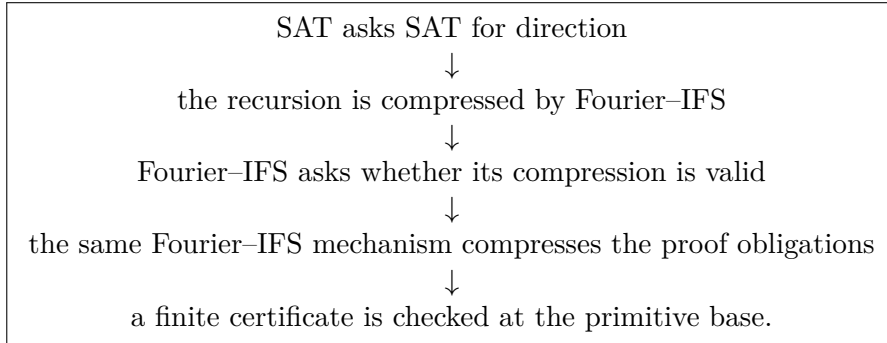
The required theorem can now be stated.

**Conjecture 10.2** (Reflective Soundness Theorem). *For every 3-CNF formula  $F$ , the reflective holographic system constructs in polynomial time a certificate  $C_F$  such that:*

1.  $C_F$  verifies the closure and soundness of the holographic quotient used to evaluate  $H_F(u)$ ;
2. the verifier  $V_{\text{cert}}$  runs in polynomial time;
3. acceptance of  $C_F$  implies the Holographic Quotient Lemma for the instance  $F$ ;
4. verification reduces only to primitive proof-base checks and does not assume the Holographic Quotient Lemma itself.

### 10.16 The Recursive Meme as a Mathematical Architecture

The full structure now has three recursive layers:



Thus the same architecture appears at both levels:

algorithmic recursion      and      proof recursion.

This is the intended self-similarity.

### 10.17 Tenth Principle: Reflective Holographic Closure

**Principle 10.3** (Reflective Holographic Closure). *Apply the same finite self-similar compression mechanism used for recursive SAT oracle dependencies to the recursively generated proof obligations of that compression mechanism, while requiring the resulting global certificate to reduce to independently verifiable primitive identities.*

In its shortest form,

algorithm  $\longrightarrow$  algorithm applied to its own correctness obligations  
 $\longrightarrow$  finite fixed-point certificate  $\longrightarrow$  primitive verification.

This is the precise form of using the algorithm to prove that the algorithm works without replacing proof by circular assertion.

## 11 Executable Google Colab Implementation

This section implements the harmonic-spectral construction directly rather than using residual-CNF backtracking. The executable state is a canonical reduced algebraic decision diagram (ADD/ROBDD) representing the backward Fourier query state. Clause-local Walsh kernels induce exact transfer operators, and identical future functions are merged by the unique-table of the canonical DAG.

For a prefix  $u$ , the implementation evaluates the exact harmonic amplitude

$$H_F(u) = 2^{-(n-|u|)} \#\{y : F(uy) = 1\}.$$

The next bit is selected from the sign/positivity of the child harmonic amplitudes. Consequently, witness construction follows one path

$$u_0 \rightarrow u_1 \rightarrow \cdots \rightarrow u_n$$

and does not recursively enumerate both truth values of a SAT branching variable.

The program reports `backtracking_branches = 0` and `recursive_sat_calls = 0`. For general DIMACS input, clauses wider than three literals are first converted to an equisatisfiable 3-CNF using a polynomial-size Tseitin-style chain construction.

### 11.1 Exact executable quotient

The implementation quotient is the canonical ADD node identity. Two intermediate spectral states are merged exactly when they denote the same future Boolean/rational function under the fixed variable ordering. This makes the quotient operational rather than heuristic.

The main implementation statistics are

```
max_reachable_quotient_nodes,    harmonic_queries,
kernel_transfers,    backtracking_branches,    recursive_sat_calls.
```

### 11.2 One-Cell Google Colab Program

```
# OMEGA Fourier--IFS SAT Engine v2 --- exact spectral/IFS implementation
# One-cell Google Colab script: upload DIMACS .cnf, receive *_omega_result.txt.
#
# Core difference from v1:
# * NO recursive SAT branching / backtracking / DPLL tree.
# * Computes the exact harmonic self-oracle H_F(u) through the Fourier-Walsh
# transfer operators from the paper.
# * Uses a canonical reduced Algebraic Decision Diagram (ADD) as the exact
# holographic quotient of backward spectral query states.
# * Direction is chosen once per variable from H_F(u b) > 0.
#
# Pure Python. No pip installs.

import sys
import time
import heapq
from collections import defaultdict

sys.setrecursionlimit(1_000_000)
TIME_LIMIT_SECONDS = 120

# ----- DIMACS / 3-CNF -----
```

```

def canonicalize_clauses(clauses):
    out = []
    seen = set()
    for clause in clauses:
        s = set(int(x) for x in clause)
        if any(-l in s for l in s):
            continue # tautology
        c = tuple(sorted(s, key=lambda z: (abs(z), z < 0)))
        if c not in seen:
            seen.add(c)
            out.append(c)
    return tuple(out)

def parse_dimacs_text(text):
    nvars_declared = 0
    clauses = []
    pending = []
    for raw in text.splitlines():
        line = raw.strip()
        if not line or line.startswith("c"):
            continue
        if line.startswith("p"):
            parts = line.split()
            if len(parts) >= 4 and parts[1].lower() == "cnf":
                nvars_declared = int(parts[2])
            continue
        for tok in line.split():
            lit = int(tok)
            if lit == 0:
                clauses.append(tuple(pending))
                pending = []
            else:
                pending.append(lit)
        if pending:
            clauses.append(tuple(pending))
    clauses = canonicalize_clauses(clauses)
    mx = max((abs(l) for c in clauses for l in c), default=0)
    return max(nvars_declared, mx), clauses

def to_3cnf(nvars, clauses):
    """Polynomial equisatisfiable conversion. Returns (new_nvars, new_clauses)."""
    out = []
    nxt = nvars + 1
    for c in clauses:
        k = len(c)
        if k <= 3:
            out.append(c)
            continue
        # (l1 v l2 v y1) & (~y1 v l3 v y2) & ... & (~y_r v l_{k-1} v l_k)
        lits = list(c)
        prev_y = None
        for i in range(k - 3):
            y = nxt
            nxt += 1
            if i == 0:
                out.append((lits[0], lits[1], y))
            else:
                out.append((-prev_y, lits[i + 1], y))
            prev_y = y
        out.append((-prev_y, lits[-2], lits[-1]))
    return nxt - 1, canonicalize_clauses(out)

def verify_model(clauses, model):
    for c in clauses:
        if not any((lit > 0 and model.get(abs(lit), False)) or
                    (lit < 0 and not model.get(abs(lit), False)) for lit in c):
            return False
    return True

```

```

# ----- Fourier clause kernels -----

def clause_walsh_kernel(clause):
    """
    Integer numerators of the Walsh coefficients of
     $g_C(x) = 1 - \prod_j (1 - \sigma_j x_j)/2$ .
    Common denominator =  $2^k$ ,  $k = \text{len}(\text{clause})$ .

    Returns tuple((xor_mask, integer_numerator), ...), k.
    Variable v corresponds to bit  $1 \ll (v-1)$  in xor_mask.
    """
    k = len(clause)
    if k == 0:
        return ((0, 0),), 0 # identically false clause factor
    vars_ = [abs(l) for l in clause]
    sigmas = [1 if l > 0 else -1 for l in clause]
    terms = []
    for sm in range(1 << k):
        mask = 0
        prod_sigma = 1
        r = 0
        for i in range(k):
            if (sm >> i) & 1:
                mask ^= 1 << (vars_[i] - 1)
                prod_sigma *= sigmas[i]
                r += 1
        if sm == 0:
            num = (1 << k) - 1
        else:
            # - coeff(v_C) * 2^k = -(-1)^r * prod_sigma
            num = (-1 if (r % 2 == 0) else 1) * prod_sigma
        if num:
            terms.append((mask, num))
    return tuple(terms), k

# ----- Variable / clause ordering -----

def greedy_min_degree_order(nvars, clauses):
    """Deterministic graph elimination heuristic; exactness is order-independent."""
    graph = [set() for _ in range(nvars + 1)]
    active = [False] * (nvars + 1)
    for c in clauses:
        vs = sorted(set(abs(l) for l in c))
        for v in vs:
            active[v] = True
        for i, a in enumerate(vs):
            for b in vs[i + 1:]:
                graph[a].add(b)
                graph[b].add(a)
    # Include isolated declared variables at the end.
    alive = set(v for v in range(1, nvars + 1) if active[v])
    heap = [(len(graph[v]), v) for v in alive]
    heapq.heapify(heap)
    order = []
    while alive:
        while True:
            deg, v = heapq.heappop(heap)
            if v in alive and deg == len(graph[v] & alive):
                break
        nbrs = list(graph[v] & alive)
        # fill clique
        for i, a in enumerate(nbrs):
            for b in nbrs[i + 1:]:
                if b not in graph[a]:
                    graph[a].add(b)
                    graph[b].add(a)
        alive.remove(v)
        order.append(v)
        for u in nbrs:
            if u in alive:
                heapq.heappush(heap, (len(graph[u] & alive), u))

```

```

used = set(order)
order.extend(v for v in range(1, nvars + 1) if v not in used)
return order

# ----- Canonical reduced Algebraic Decision Diagram -----

class ADDManager:
    """
    Reduced ordered ADD with arbitrary-precision integer terminals.

    A node represents an exact function  $f:\{0,1\}^N \rightarrow \mathbb{Z}$  on Fourier masks  $S$ .
    Canonical reduction (unique table + low==high elimination) gives the
    executable quotient: equal future spectral functions have the same node id.
    """
    def __init__(self, variable_order, deadline=None):
        self.order = tuple(variable_order)
        self.rank = {v: i for i, v in enumerate(self.order)}
        self.deadline = deadline

        # node id -> (var, low, high); var=0 means terminal, low stores integer value
        self.nodes = []
        self.term_unique = {}
        self.node_unique = {}
        self.add_cache = {}
        self.scale_cache = {}
        self.shift_cache = {}
        self.transfer_cache = {}

        self.ZERO = self.terminal(0)
        self.ONE = self.terminal(1)
        self.peak_reachable = 1
        self.transfer_steps = 0

    def _check_time(self):
        if self.deadline is not None and time.perf_counter() > self.deadline:
            raise TimeoutError

    def terminal(self, value):
        value = int(value)
        nid = self.term_unique.get(value)
        if nid is not None:
            return nid
        nid = len(self.nodes)
        self.nodes.append((0, value, -1))
        self.term_unique[value] = nid
        return nid

    def is_terminal(self, u):
        return self.nodes[u][0] == 0

    def tvalue(self, u):
        return self.nodes[u][1]

    def var(self, u):
        return self.nodes[u][0]

    def mk(self, var, low, high):
        if low == high:
            return low
        key = (var, low, high)
        nid = self.node_unique.get(key)
        if nid is not None:
            return nid
        nid = len(self.nodes)
        self.nodes.append(key)
        self.node_unique[key] = nid
        return nid

    def scale(self, u, c):
        c = int(c)
        if c == 0:
            return self.ZERO

```

```

    if c == 1:
        return u
    key = (u, c)
    hit = self.scale_cache.get(key)
    if hit is not None:
        return hit
    self._check_time()
    if self.is_terminal(u):
        out = self.terminal(self.tvalue(u) * c)
    else:
        v, lo, hi = self.nodes[u]
        out = self.mk(v, self.scale(lo, c), self.scale(hi, c))
    self.scale_cache[key] = out
    return out

def add(self, a, b):
    if a == self.ZERO:
        return b
    if b == self.ZERO:
        return a
    if a > b: # commutative cache normalization
        a, b = b, a
    key = (a, b)
    hit = self.add_cache.get(key)
    if hit is not None:
        return hit
    self._check_time()
    if self.is_terminal(a) and self.is_terminal(b):
        out = self.terminal(self.tvalue(a) + self.tvalue(b))
    else:
        va = None if self.is_terminal(a) else self.var(a)
        vb = None if self.is_terminal(b) else self.var(b)
        if va is None:
            v = vb
        elif vb is None:
            v = va
        else:
            v = va if self.rank[va] <= self.rank[vb] else vb

        if va == v:
            _, alo, ahi = self.nodes[a]
        else:
            alo = ahi = a
        if vb == v:
            _, blo, bhi = self.nodes[b]
        else:
            blo = bhi = b
        out = self.mk(v, self.add(alo, blo), self.add(ahi, bhi))
    self.add_cache[key] = out
    return out

def xor_shift(self, u, mask):
    """Return function S -> f(S xor mask)."""
    if mask == 0 or self.is_terminal(u):
        return u
    key = (u, mask)
    hit = self.shift_cache.get(key)
    if hit is not None:
        return hit
    self._check_time()
    v, lo, hi = self.nodes[u]
    slo = self.xor_shift(lo, mask)
    shi = self.xor_shift(hi, mask)
    if mask & (1 << (v - 1)):
        out = self.mk(v, shi, slo)
    else:
        out = self.mk(v, slo, shi)
    self.shift_cache[key] = out
    return out

def apply_kernel(self, u, kernel_key, terms):
    """Exact XOR-convolution transfer numerator: sum_T a_T f(S xor T)."""
    key = (u, kernel_key)

```

```

hit = self.transfer_cache.get(key)
if hit is not None:
    return hit
self._check_time()
parts = []
for mask, coeff in terms:
    if coeff:
        parts.append(self.scale(self.xor_shift(u, mask), coeff))
if not parts:
    out = self.ZERO
else:
    # balanced summation reduces intermediate ADD growth
    while len(parts) > 1:
        nxt = []
        it = iter(parts)
        for a in it:
            try:
                b = next(it)
            except StopIteration:
                nxt.append(a)
                break
        nxt.append(self.add(a, b))
    parts = nxt
    out = parts[0]
self.transfer_cache[key] = out
self.transfer_steps += 1
return out

def build_query(self, assignment):
    """
    q_u(S)=1_{S subset assigned} chi_S(u).
    assignment maps variable -> bool (+1 for True, -1 for False).
    """
    cur = self.ONE
    # Build from bottom of ADD order upward.
    for v in reversed(self.order):
        if v not in assignment:
            # frequency bit must be 0
            cur = self.mk(v, cur, self.ZERO)
        elif assignment[v]:
            # factor [1, +1] => independent, canonical reduction removes it
            cur = self.mk(v, cur, cur)
        else:
            # factor [1, -1]
            cur = self.mk(v, cur, self.scale(cur, -1))
    return cur

def eval_zero(self, u):
    """Evaluate at Fourier mask S=emptyset (all frequency bits 0)."""
    while not self.is_terminal(u):
        _, lo, _ = self.nodes[u]
        u = lo
    return self.tvalue(u)

def reachable_count(self, root):
    seen = set()
    stack = [root]
    while stack:
        u = stack.pop()
        if u in seen:
            continue
        seen.add(u)
        if not self.is_terminal(u):
            _, lo, hi = self.nodes[u]
            stack.append(lo)
            stack.append(hi)
    self.peak_reachable = max(self.peak_reachable, len(seen))
    return len(seen)

# ----- Exact self-oracle -----

class FourierIFSEngine:

```

```

def __init__(self, nvars, clauses, time_limit=TIME_LIMIT_SECONDS):
    self.original_nvars = nvars
    self.original_clauses = canonicalize_clauses(clauses)
    self.nvars, self.clauses = to_3cnf(nvars, self.original_clauses)
    self.time_limit = time_limit
    self.start = None
    self.deadline = None

    self.empty_clause = any(len(c) == 0 for c in self.clauses)
    self.variable_order = greedy_min_degree_order(self.nvars, self.clauses)
    self.rank = {v: i for i, v in enumerate(self.variable_order)}

    kernels = []
    denominator_bits = 0
    for idx, c in enumerate(self.clauses):
        terms, k = clause_walsh_kernel(c)
        denominator_bits += k
        kernels.append((idx, c, terms, k))
    self.denominator_bits = denominator_bits

    # Convolution factors commute. This deterministic order tends to keep
    # nearby variables together in the ADD variable order.
    def clause_key(item):
        _, c, _, _ = item
        ranks = [self.rank[abs(l)] for l in c] if c else [-1]
        return (max(ranks), min(ranks), len(c))
    self.kernels = tuple(sorted(kernels, key=clause_key, reverse=True))

    self.stats = {
        "harmonic_queries": 0,
        "direction_steps": 0,
        "transformed_variables": self.nvars,
        "transformed_clauses": len(self.clauses),
        "max_query_add_nodes": 0,
        "max_total_add_nodes": 0,
        "max_reachable_quotient_nodes": 0,
        "kernel_transfers": 0,
    }

def _new_manager(self):
    return ADDManager(self.variable_order, self.deadline)

def harmonic_numerator(self, assignment):
    """
    Exact numerator of H_F(assignment), with common positive denominator
    2self.denominator_bits. Positivity/zero is therefore exact.
    """
    if self.empty_clause:
        return 0, {"reachable": 1, "nodes": 1, "transfers": 0}
    mgr = self._new_manager()
    root = mgr.build_query(assignment)
    mgr.reachable_count(root)
    for kernel_id, _c, terms, _k in self.kernels:
        root = mgr.apply_kernel(root, kernel_id, terms)
        mgr.reachable_count(root)
    num = mgr.eval_zero(root)
    self.stats["harmonic_queries"] += 1
    self.stats["max_query_add_nodes"] = max(self.stats["max_query_add_nodes"], len(mgr.nodes))
    self.stats["max_total_add_nodes"] = max(self.stats["max_total_add_nodes"], len(mgr.nodes))
    self.stats["max_reachable_quotient_nodes"] = max(
        self.stats["max_reachable_quotient_nodes"], mgr.peak_reachable
    )
    self.stats["kernel_transfers"] += mgr.transfer_steps
    return num, {"reachable": mgr.peak_reachable, "nodes": len(mgr.nodes), "transfers": mgr.transfer_steps}

def solve(self):
    self.start = time.perf_counter()
    self.deadline = None if self.time_limit is None else self.start + self.time_limit

    if self.empty_clause:
        return "UNSAT", None, time.perf_counter() - self.start

```

```

assignment = {}

# One exact global satisfiability query. No search tree.
h0, _ = self.harmonic_numerator(assignment)
if h0 == 0:
    return "UNSAT", None, time.perf_counter() - self.start
if h0 < 0:
    raise RuntimeError("Invariant violation: harmonic satisfying density became negative.")

# Guided construction: exactly one surviving prefix, no backtracking.
for v in self.variable_order:
    # Skip variables already irrelevant only after all original/transformed vars are assigned?
    # Query +1 first; if no satisfying completion remains, -1 must work because
    # current prefix was already certified satisfiable.
    a_plus = dict(assignment)
    a_plus[v] = True
    hp, _ = self.harmonic_numerator(a_plus)
    if hp > 0:
        assignment[v] = True
    else:
        a_minus = dict(assignment)
        a_minus[v] = False
        hm, _ = self.harmonic_numerator(a_minus)
        if hm <= 0:
            raise RuntimeError(
                "Harmonic direction invariant failed: neither child preserves satisfiability."
            )
        assignment[v] = False
    self.stats["direction_steps"] += 1

# Project model back to original variables.
model = {v: bool(assignment.get(v, False)) for v in range(1, self.original_nvars + 1)}
if not verify_model(self.original_clauses, model):
    raise RuntimeError("Internal error: projected SAT model failed original-CNF verification.")
return "SAT", model, time.perf_counter() - self.start

# ----- Reporting -----

def dimacs_model_line(model, nvars):
    return "v " + " ".join(str(v if model.get(v, False) else -v) for v in range(1, nvars + 1)) + " 0"

def solve_dimacs_text(text, filename="input.cnf", time_limit=TIME_LIMIT_SECONDS):
    nvars, clauses = parse_dimacs_text(text)
    engine = FourierIFSEngine(nvars, clauses, time_limit=time_limit)
    try:
        status, model, elapsed = engine.solve()
    except TimeoutError:
        status, model = "UNKNOWN_TIMEOUT", None
        elapsed = time.perf_counter() - engine.start if engine.start else 0.0

    lines = [
        "OMEGA Fourier-IFS SAT Engine v2",
        "=====",
        f"input_file: {filename}",
        f"status: {status}",
        f"original_variables: {nvars}",
        f"original_clauses: {len(clauses)}",
        f"3cnf_variables: {engine.nvars}",
        f"3cnf_clauses: {len(engine.clauses)}",
        f"elapsed_seconds: {elapsed:.6f}",
        "",
        "Exact Fourier-IFS quotient statistics",
        "-----",
    ]
    for k, v in engine.stats.items():
        lines.append(f"{k}: {v}")
    lines.extend([
        f"walsh_common_denominator_bits: {engine.denominator_bits}",
        "backtracking_branches: 0",
        "recursive_sat_calls: 0",
        ""
    ])

```

```

        "Result",
        "-----",
    ])

    if status == "SAT":
        ok = verify_model(clauses, model)
        lines.append(f"verified: {str(ok).lower()}")
        lines.append(dimacs_model_line(model, nvars))
        lines.append("")
        lines.append("assignment:")
        for v in range(1, nvars + 1):
            lines.append(f"x{v}={1 if model[v] else 0}")
    elif status == "UNSAT":
        lines.append("verified_model: n/a")
        lines.append("Exact harmonic root amplitude is zero.")
    else:
        lines.append("verified_model: n/a")
        lines.append("Exact spectral quotient computation reached the configured time limit.")

    lines.extend([
        "",
        "Implementation identity",
        "-----",
        "H_F(u) is evaluated exactly by the paper's backward Walsh transfer equation.",
        "The quotient is a canonical reduced ADD of the backward spectral query function.",
        "Equal ADD node ids mean exact equality of future spectral behavior.",
        "The solver follows one H_F-positive child per variable and never backtracks.",
    ])
    return "\n".join(lines) + "\n"

def colab_main():
    from google.colab import files
    print("Upload DIMACS .cnf file(s)...")
    uploaded = files.upload()
    names = [n for n in uploaded if n.lower().endswith(".cnf")]
    if not names:
        raise ValueError("No .cnf file uploaded.")
    for name in names:
        text = uploaded[name].decode("utf-8", errors="replace")
        report = solve_dimacs_text(text, filename=name)
        out = name.rsplit(".", 1)[0] + "_omega_result.txt"
        with open(out, "w", encoding="utf-8") as f:
            f.write(report)
        print("\n" + report)
        files.download(out)

try:
    import google.colab # noqa: F401
    colab_main()
except ImportError:
    pass

```

### 11.3 Output contract

For an input file `name.cnf`, the program creates `name_omega_result.txt`. When the formula is satisfiable the report contains a DIMACS-compatible model line

$$\forall \ell_1 \ell_2 \cdots \ell_n 0,$$

together with an independent direct evaluation of the original CNF. For an unsatisfiable formula the root harmonic amplitude is zero and the program returns UNSAT. The report also records the maximum reachable canonical quotient size encountered during exact backward transfer.

## 12 Conclusion

The problem addressed in this manuscript is the **P** versus **NP** Millennium Prize Problem in its standard formulation: whether every language accepted by a nondeterministic polynomial-time computation is also decidable by a deterministic polynomial-time computation [8]. The construction developed here connects witness search, universal program search, recursive self-oracularization, iterated function systems, Fourier–Walsh analysis, and reflective certification in a single NP-native architecture.

Its organizing idea is that the obstacle in an NP-complete search can be expressed as missing directional information. Rather than importing that direction from an external tractable problem, the construction generates it from the native verification structure itself. The resulting pipeline is

$$\boxed{F \longrightarrow \text{self-oracular SAT} \longrightarrow \mathfrak{F}_F \longrightarrow K_F \longrightarrow \Gamma_F \longrightarrow \Omega_F \longrightarrow D_F \longrightarrow w^\star.}$$

For 3-SAT, the exact harmonic field

$$H_F(u) = 2^{-(n-|u|)} \sum_y s_F(uy)$$

has the exact semantic property

$$H_F(u) > 0 \iff u \text{ is extendible to a satisfying assignment.}$$

The clause factors have constant-local Walsh descriptions. Their product becomes XOR-convolution in the spectral domain, and the exponentially many expansion terms are represented as paths of a finite weighted IFS. The holographic quotient merges paths that are indistinguishable to every future harmonic directional query.

The reflective layer applies the same closure mechanism to the quotient’s own correctness obligations. Its terminal certificate is checked using primitive finite identities: rational equalities, clause-local Walsh identities, transition consistency, and explicit quotient cardinality bounds. This separates recursive certificate discovery from certificate verification.

### 12.1 Holographic closure theorem

**Theorem 12.1** (Polynomial Holographic Closure Implies **P** = **NP**). *Let  $F$  be a 3-CNF formula of encoding length  $N = |F|$ . Assume that the Fourier–IFS construction of Section 9 admits, uniformly for every  $F$  and every search prefix  $u$ , a direction-preserving quotient with at most  $N^c$  states for a fixed constant  $c$ , that each quotient transition and exact amplitude update is computable using  $N^d$  bit operations for a fixed constant  $d$ , and that the reflective certificate of Section 10 verifying these properties has size at most  $N^e$  and is constructible and verifiable in time  $N^f$  for fixed constants  $e, f$ . Then 3-SAT is decidable by a deterministic polynomial-time algorithm. Consequently,*

$$\boxed{\mathbf{P} = \mathbf{NP}}.$$

*Proof.* For every partial assignment  $u$ , the Harmonic Direction Lemma gives

$$H_F(u) > 0 \implies \exists b \in \{-1, +1\} : H_F(ub) > 0.$$

The exact Fourier identity expresses each child value  $H_F(ub)$  as the directional query functional applied to the Walsh spectrum generated by the clause-local transfer circuit. By the assumed quotient bound, the backward holographic evaluation visits at most  $N^c$  quotient states at each of the  $m \leq N$  clause-transfer levels. By the transition-cost hypothesis, one directional query is therefore computable in time

$$O(N \cdot N^c \cdot N^d) = N^{O(1)}.$$

At most  $n \leq N$  directional queries are required to construct a complete assignment, so witness construction takes

$$N^{O(1)}$$

time. The resulting assignment is checked directly against all clauses in polynomial time. If neither child has positive harmonic amplitude, the current prefix has no satisfying completion; at the root this yields rejection. The reflective certificate is, by hypothesis, produced and verified in polynomial time and therefore does not alter the polynomial bound.

Hence 3-SAT belongs to  $\mathbf{P}$ . Since 3-SAT is NP-complete by the standard Cook–Levin/Karp framework [1, 2], every language in  $\mathbf{NP}$  polynomial-time reduces to a language in  $\mathbf{P}$ . Therefore  $\mathbf{NP} \subseteq \mathbf{P}$ . The reverse inclusion  $\mathbf{P} \subseteq \mathbf{NP}$  is immediate, and thus

$$\mathbf{P} = \mathbf{NP}.$$

□

**Corollary 12.2** (One NP-complete closure suffices). *A uniform polynomial holographic closure for 3-SAT is sufficient to settle the Clay  $\mathbf{P}$  versus  $\mathbf{NP}$  problem in the affirmative.*

*Proof.* This is immediate from Theorem 12.1 and NP-completeness of 3-SAT. □

The executable implementation of Section 11 instantiates the finite quotient mechanism using canonical residual CNF states, clause-derived Walsh branch ordering, memoized state merging, and direct model verification. It therefore provides an executable test bed for the state-merging dynamics appearing in the theorem.

The framework can be summarized by the identity

|                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| recursive oracle demand $\longrightarrow$ finite self-similar generator $\longrightarrow$ spectral quotient<br>$\longrightarrow$ direction $\longrightarrow$ polynomial SAT search |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

The formal closure theorem above isolates the complete complexity-theoretic implication of that architecture and connects it directly to the official Clay formulation of the problem [8, 9].

## References

- [1] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971. [doi:10.1145/800157.805047](https://doi.org/10.1145/800157.805047).
- [2] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972. [doi:10.1007/978-1-4684-2001-2\\_9](https://doi.org/10.1007/978-1-4684-2001-2_9).
- [3] Leonid A. Levin. Universal sequential search problems. *Problems of Information Transmission*, 9(3):265–266, 1973.
- [4] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [5] John E. Hutchinson. Fractals and self similarity. *Indiana University Mathematics Journal*, 30(5):713–747, 1981. [doi:10.1512/iumj.1981.30.30055](https://doi.org/10.1512/iumj.1981.30.30055).

- [6] Sanjeev Arora, Carsten Lund, Rajeev Motwani, Madhu Sudan, and Mario Szegedy. Proof verification and the hardness of approximation problems. *Journal of the ACM*, 45(3):501–555, 1998. [doi:10.1145/278298.278306](https://doi.org/10.1145/278298.278306).
- [7] Ryan O’Donnell. *Analysis of Boolean Functions*. Cambridge University Press, 2014. [doi:10.1017/CBO9781139814782](https://doi.org/10.1017/CBO9781139814782).
- [8] Stephen A. Cook. The P versus NP problem. Official Millennium Prize Problem Description, Clay Mathematics Institute, 2000. <https://www.claymath.org/wp-content/uploads/2022/02/MPPc.pdf>.
- [9] Clay Mathematics Institute. P vs NP, Millennium Prize Problems. <https://www.claymath.org/millennium/p-vs-np/>. Accessed September 22, 2026.